TopoGraph: teaching LLMs to draw the dataflow graph of PyTorch code

An explainer of the whole project: data, code, journey, results, examples and caveats

IE 643 (IIT Bombay, Autumn 2026)
Team MachineUnlearning: Devanshu Jha & Peela Rathna Harsha
Written from the project repository (code, logs and result files in `IE643_Project/topograph`)

25 October 2026

The project in one paragraph

Given the source code of a PyTorch `nn.Module`, a large language model (LLM) must write down the *forward-pass dataflow graph* of that module: one node per layer or operation, one edge per tensor that flows between them, plus each node's kind and output tensor shape. The exact answer is computed by `torch.fx`, so we can grade every prediction automatically. We built the full pipeline (data splits, a reference "oracle", a validator that checks a graph *without* knowing the answer, many metrics, a graph drawer, LoRA fine-tuning, fast inference with constrained decoding and self-correction, an evaluation suite and a demo app) and ran 30+ experiments with Qwen3.5 (0.8B–27B) and Qwen2.5-Coder-7B. **Main result:** fine-tuning makes a 9B model essentially perfect on the dataset's own distribution (99.7% valid, 100% on held-out topologies), but it breaks on genuinely new code (47.5% on our hand-written out-of-distribution set). Adding ∼2,800 randomly generated, automatically labelled training modules fixes most of that (95%; 97.5% with validator-driven retry), while a 27B general model with retry is the most robust on code that uses operators nobody trained on.

Contents

1 The task, with one complete example **4**

2 The data **5**
2.1 The given dataset (5,000 samples) . . . 5
2.2 The evaluation sets we added . . . 6
2.3 Synthetic training data (the "aug" experiment) . . . 6

3 How the system works (and where the code is) **7**
3.1 The validator: checking a graph without knowing the answer . . . 8
3.2 Metrics: how close is a prediction to the reference? . . . 8
3.3 Models, fine-tuning and inference . . . 8

4 The journey: what we did, what broke, what we changed **9**

5 Results **10**
5.1 The main table . . . 10
5.2 Complexity: which architectures are hard? . . . 11
5.3 Memorisation versus generalisation . . . 11
5.4 Robustness, validity and self-correction . . . 12
5.5 Failure modes, properties, metric check, training . . . 13

6 Twelve worked examples, from code to graph **14**
6.1 Example 1: A neuron-level MLP (tier L0) . . . 15
6.2 Example 2: A linear chain with dead code (tier L1) . . . 16
6.3 Example 3: A residual network (tier L2): the missing skip edge . . . 17
6.4 Example 4: Parallel branches joined by concat (tier L3) . . . 19
6.5 Example 5: A held-out nested topology (tier L4): memorisation vs generalisation . . . 20
6.6 Example 6: The zero-shot runaway loop . . . 22
6.7 Example 7: OOD: nested submodules (dotted scopes) . . . 24
6.8 Example 8: OOD: an nn.ModuleList loop . . . 26
6.9 Example 9: OOD: Squeeze-and-Excitation, fixed by verify-and-retry . . . 27
6.10 Example 10: OOD-2: a transposed-convolution decoder (shape failure) . . . 28
6.11 Example 11: OOD-2: attention with matmul and softmax . . . 29
6.12 Example 12: Untraceable code: no torch.fx reference exists . . . 30

7 Findings **31**

8 Caveats and limitations **31**

9 Reproducing and exploring **32**

A Glossary **32**

B Complete results **34**
B.1 Every inference configuration . . . 34
B.2 Official test: every headline metric with confidence intervals . . . 35
B.3 Every configuration on every evaluation set . . . 35
B.4 By complexity tier (official test) . . . 36
B.5 By graph size (official test) . . . 37
B.6 By augmentation variant (robustness to renaming and dead code) . . . 37
B.7 Seen versus held-out topologies . . . 37
B.8 OOD-1 and OOD-2 by category . . . 38
B.9 Validity levels and code-consistency rates . . . 38
B.10 Verify-and-retry, every run . . . 39
B.11 Topological properties preserved (official test) . . . 40
B.12 Failure labels (official test) . . . 40
B.13 Checking the edit-distance metric against the true graph edit distance . . . 40
B.14 Generation statistics: prompt and answer lengths, truncation, speed . . . 41
B.15 Determinism and small inconsistencies . . . 42

C Data details **42**
C.1 The given dataset, per tier . . . 42
C.2 Splits and the novel-topology holdout . . . 43
C.3 Node kinds, token budget, few-shot examples . . . 43
C.4 OOD-1, OOD-2 and the untraceable modules . . . 43
C.5 The synthetic data generator . . . 45

D Prompts and output format **45**
D.1 The system prompt (identical for every model, training and inference) . . . 45
D.2 User message, few-shot layout, retry message . . . 45
D.3 The JSON schema used for constrained decoding . . . 46

E Validator and metric reference **46**
E.1 Every validator check . . . 46
E.2 Metric definitions . . . 47

F Training and inference settings **48**

G Software, hardware and compute **49**

H Decision log **50**

I Problems encountered and how they were fixed **51**

J Timeline **52**

K Repository inventory and tests **52**

L The demo app **54**

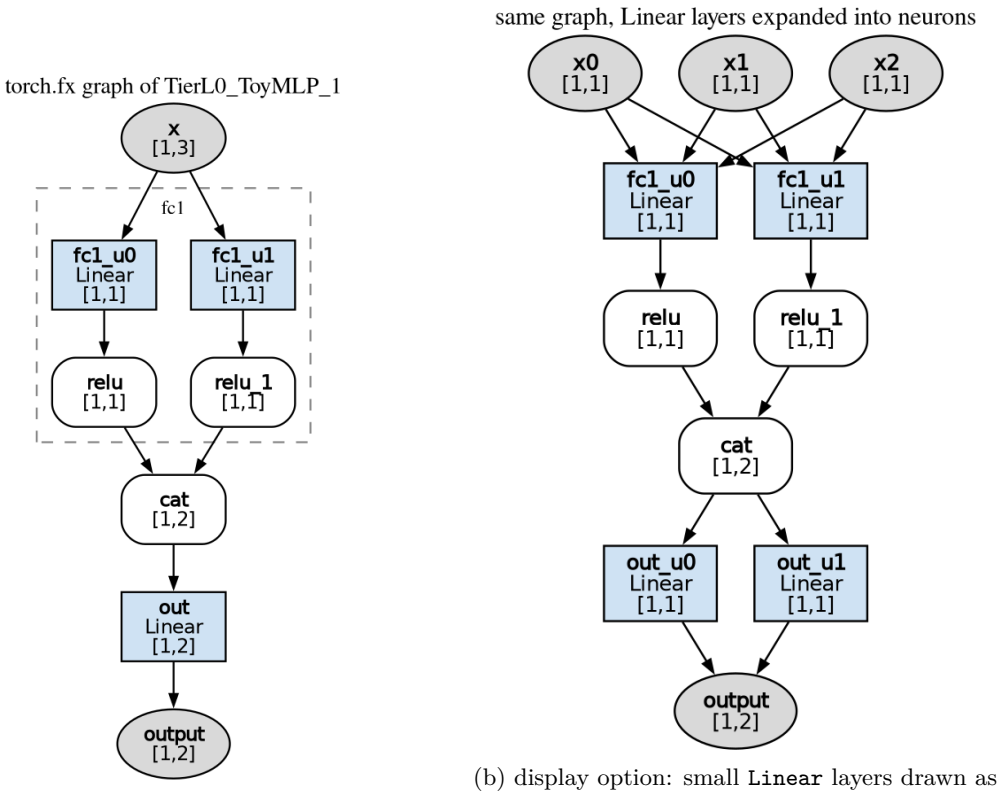
1 The task, with one complete example

Take this small network from the dataset (sample `L0_0000`):

```
class TierL0_ToyMLP_1(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1_u0 = nn.Linear(3, 1)
        self.fc1_u1 = nn.Linear(3, 1)
        self.out = nn.Linear(2, 2)

    def forward(self, x):
        u0 = torch.relu(self.fc1_u0(x))
        u1 = torch.relu(self.fc1_u1(x))
        h = torch.cat([u0, u1], dim=1)
        y = self.out(h)
        return y
```

PyTorch's `torch.fx.symbolic_trace` runs `forward` with placeholder values and records every operation. After removing dead code and propagating shapes, it yields the graph in Figure 1: an input node (`placeholder`), one node per executed layer (`call_module`) or function (`call_function`), and one `output` node. Edges say "the output of u is an input of v".



(a) the reference graph

(b) display option: small `Linear` layers drawn as neurons

Figure 1: The torch.fx graph of the module above, drawn by our visualiser. Blue boxes are layers (`call_module`: name, class, output shape); white rounded boxes are functions; grey ellipses are the input and output. The dashed box is a "hierarchy cluster": layers that share a name prefix (`fc1_`).

What the model has to output. The LLM writes the same graph in a compact JSON format we call **CG-JSON** ("compact graph JSON"): a list of nodes (`name`, `op`, `kind`, `shape`) and a list of edges as index pairs into that list. For the example above the full answer is:

```
{"nodes": [{"name": "x", "op": "placeholder", "kind": "placeholder", "shape": [1, 3]}, {"name": "fc1_u0", "op":
    "call_module", "kind": "Linear", "shape": [1, 1]}, {"name": "relu", "op": "call_function", "kind": "relu",
    "shape": [1, 1]}, {"name": "fc1_u1", "op": "call_module", "kind": "Linear", "shape": [1, 1]}, {"name": "
    relu_1", "op": "call_function", "kind": "relu", "shape": [1, 1]}, {"name": "cat", "op": "call_function", "
    kind": "cat", "shape": [1, 2]}, {"name": "out", "op": "call_module", "kind": "Linear", "shape": [1, 2]}, {"
```

```
name": "output", "op": "output", "kind": "output", "shape": [1, 2]}], "edges": [[0, 1], [1, 2], [0, 3], [3,
 4], [2, 5], [4, 5], [5, 6], [6, 7]]}
```


Why this is a good test of an LLM. The model must (i) find which layers are actually used and in what order, (ii) follow variables through the code to get the wiring right, including long *skip connections* and merges, (iii) know PyTorch naming conventions (`relu`, `relu_1`, ...), and (iv) do arithmetic on tensor shapes. Because `torch.fx` gives the exact answer, everything can be scored automatically. `torch.fx` is therefore both the *label source* and the *upper bound* (it scores 100% on every traceable set by construction). The LLM is interesting where tracing is impossible (data-dependent `if`s, incomplete code) and as a study of how well LLMs reason about graph structure.

2 The data

2.1 The given dataset (5,000 samples)

Our teammate built the dataset: 5,000 JSON records, each with `source_code` (exactly one `nn.Module` class) and its `target_graph` from `torch.fx`. It is organised in five *complexity tiers*:

Tier	What the architectures look like	nodes (min/med/max)	distinct topologies
L0	toy MLP where every hidden unit is its own `Linear(k,1)` ("neurons")	8 / 10 / 14	6
L1	linear chain of conv / batch-norm / activation / dropout	6 / 15 / 26	21
L2	residual blocks (skip connection, optional projection)	9 / 17 / 26	14
L3	multiple parallel branches merged by `torch.cat`	8 / 12 / 20	12
L4	nested: stem → 2 paths → concat → merge → residual add → head	15 / 17 / 19	4

There are 1,250 *families* (one base architecture each), and every family appears in four variants with an *identical* graph: the original, the forward argument renamed (e.g. `feat_map`), dead code inserted (`self.unused_op_K = nn.Identity()` that is called but never used), and all three changes plus a shuffled `__init__`. The official split is 4,000 train / 1,000 test, separated by family. Inputs are always batch 1 and 16×16 images, so shapes are predictable from the code (Figure 2).

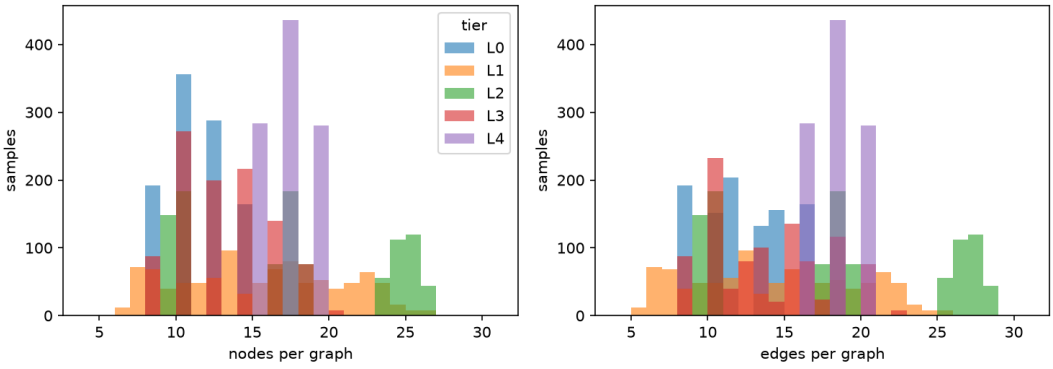


Figure 2: Graph sizes per tier (all 5,000 samples).

Three weaknesses of the dataset we had to design around

1. **Very few topologies.** Ignoring names, there are only **54 distinct graph structures** in all 5,000 samples (L4 has 4), and every structure in the official test also occurs in training. The official test therefore measures "new code, *seen* topology". A model could score highly by memorising templates.
2. **Class names leak the tier** (`TierL2_Residual_17`).
3. **Everything is flat**: no nested submodule classes, no `ModuleList`, no pooling in most tiers.

2.2 The evaluation sets we added

- **test** (1,000): the official test, unchanged. 200 of its samples happen to have a structure we held out (below), so we can split it into "seen" and "held-out" topologies.
- **novel_topo** (564): before any training we removed 11 of the topology classes (1–4 per tier, chosen with a fixed random seed, never the largest) from the training data entirely. `novel_topo` contains official-train samples with those structures, so the fine-tuned models have never seen these graph shapes.
- **test_anon** (1,000): the official test with every class renamed to `Net`.
- **OOD-1** (40): hand-written "out-of-distribution" modules in 10 categories: MLPs with LayerNorm/GELU, VGG-style with pooling, nested ResNet blocks (submodule classes), bottlenecks with `F.relu`, Inception branches, U-Net with up-sampling, DenseNet concatenations, Squeeze-and-Excitation (tensor methods + broadcasting multiply), two-input models, and `ModuleList`/`Sequential` loops.
- **OOD-2** (21): written *after* the synthetic generator (Section 2.3) was frozen, using constructs that occur in neither the dataset nor the generator: functional pooling, `view`/`reshape`/ `permute`, attention with `matmul`+`softmax`, transposed convolutions, GroupNorm/PReLU/ELU/Hardswish, `torch.stack`, scalar arithmetic. It is the most honest generalisation test we have.
- **untraceable** (6): modules `torch.fx` cannot trace (data-dependent `if`/`while`, `assert`, `int(x.shape)`): no reference, used only to show what models do.
- **val** (160) and **test_mini** (100, stratified) for training sanity checks and expensive experiments.

Fine-tuning used **train_ft** = 3,068 samples (official train minus the held-out structures minus val). Nothing from any evaluation set was ever used for training, prompts or model selection.

2.3 Synthetic training data (the "aug" experiment)

To attack the "54 topologies" problem we wrote a generator (`scripts/gen_synthetic.py`) that composes random architectures from general PyTorch building blocks (residual blocks, branch-concat, pooling, U-Net skips, dense concatenations, gating, nested block classes, `Sequential`, `ModuleList` loops, two-input fusion, MLPs), with random (sometimes deliberately misleading) attribute names. Every module is labelled by the same oracle and kept only if the validator accepts the gold graph. We generated 3,200 modules (2,090 distinct structures); 2,809 fit the token budget and were added to train_ft. **Leakage guard:** any generated module whose structure equals a `novel_topo` or OOD-1 structure was discarded (264 were); a separate audit found zero overlap.

3 How the system works (and where the code is)

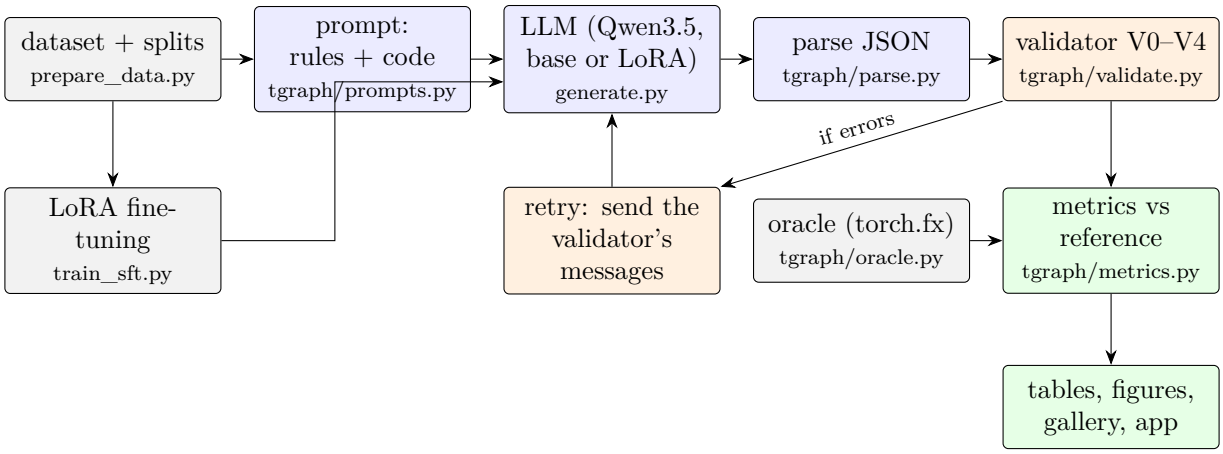


Figure 3: The pipeline. The validator never sees the answer, so it can be used at inference time (retry) and in the app; the oracle is used only for scoring.

File	What it does
`tgraph/oracle.py`	runs `torch.fx`: trace, remove dead code, propagate shapes. Reproduces all 5,000 labels byte-for-byte (a test checks this). A sandboxed version runs user code in a separate process with a 20 s limit.
`tgraph/schema.py`	converts between the dataset format and CG-JSON; the JSON schema used for constrained decoding.
`tgraph/prompts.py`	the system prompt (8 rules: node order, naming, `call_module` vs `call_function`, dead code, shape convention, edge format), chat rendering, few-shot and retry messages.
`tgraph/parse.py`	extracts JSON from raw model text (removes `<think>` blocks and code fences, finds the first balanced `{...}`, repairs truncated JSON).
`tgraph/codeinfo.py`	reads the source with Python's `ast` (never executes it): which layers `__init__` declares, with which arguments, and which of them actually reach the `return` (a backward "liveness" pass).
`tgraph/validate.py`	the 5-level checker (below).
`tgraph/align.py`, `metrics.py`, `properties.py`, `failures.py`	node matching, all metrics, topological properties, failure labels.
`tgraph/visualize.py`	Graphviz drawings: single graph, side-by-side, colour-coded diff, neuron expansion; matplotlib fallback.
`scripts/`	`prepare_data`, `build_ood`, `gen_synthetic`, `train_sft`, `merge_lora`, `generate`, `run_eval`, `aggregate`, `plots`, `gallery`, ...
`app/`	Gradio demo: paste code → predicted graph, reference, diff, metrics, validity report.
`configs/ experiments.yaml`	every inference configuration (model, prompt mode, decoding, eval sets).

3.1 The validator: checking a graph without knowing the answer

Level	Checks
V0 parse	the output is JSON and follows the schema
V1 structure	unique names; edges point to existing nodes; no self-loops/duplicates; one input, one output at the end; no cycles; nodes listed in execution order; every node reachable from the input and leading to the output
V2 arity	`add`/`mul` need two tensor inputs, `cat` at least two, a `Conv2d` exactly one, ...
V3 code consistency	every `call_module` node is a layer declared in `__init__` (otherwise "hallucinated"), with the declared class; every used layer appears as often as it is called; dead layers do not appear
V4 shapes	e.g. a `Conv2d`'s channel count equals its `out_channels`; activations keep the shape; `add` inputs broadcast; `cat` sums channels; the input shape follows the convention

A graph is **fully valid** if it passes V0–V4. The validator has zero false alarms on all 5,000 gold graphs and on all 61 OOD gold graphs, and catches every synthetic corruption in the tests. Its messages are short English sentences (e.g. `node 15 'add' has 1 input; 'add' of two tensors needs 2 (missing skip edge?)`), which is what makes *verify-and-retry* possible: failing graphs are sent back to the model together with these messages.

3.2 Metrics: how close is a prediction to the reference?

- **Node / edge F1**: nodes are matched by name; an edge is correct if the same named pair is connected.
- **Exact match**: same nodes (name, op, kind) and same edge set. **Isomorphic**: same graph up to renaming (only structure and kinds count).
- **Graph similarity** = 1− normalised edit distance (how many nodes/edges must change). We validated this cheap distance against the true graph edit distance on 1,378 pairs: Spearman correlation **0.998** (Figure 11).
- **Property preservation (PPS)**: share of 14 topological properties that match (depth, width, number of branches/merges/skip edges/paths, ...). **Shape accuracy**: share of matched nodes with the right shape.
- Every number comes with a bootstrap 95% confidence interval; results are sliced by tier, size, augmentation, seen/held-out topology and OOD category.

3.3 Models, fine-tuning and inference

- **Models.** Qwen3.5-2B and -9B (the 9B was already on the cluster), Qwen3.5-27B in FP8 as the "ceiling", Qwen2.5-Coder-7B as an older code-specialised baseline. Qwen3.5 mixes normal attention with "Gated DeltaNet" linear-attention layers, which needs special GPU kernels (`flash-linear-attention`).
- **Prompting.** Zero-shot (rules + code) and 3-shot (plus three solved training examples).
- **Fine-tuning** (LoRA, bf16, 2 epochs): small trainable matrices added to every attention, MLP and linear-attention projection; the loss is computed only on the answer tokens. Training is short: 2B 0.5 h, 9B 1.5 h (3 h with synthetic data) on one NVIDIA L40S. Training and inference use byte-identical prompts.
- **Inference** with vLLM, greedy decoding. *Constrained decoding* forces the output to match the JSON schema. *Verify-and-retry*: up to 2 extra rounds with the validator's messages. *Self-consistency*: 5 samples, the validator picks. *Thinking mode*: Qwen3.5 reasons in a `<think>` block before answering.

4 The journey: what we did, what broke, what we changed

1. Setting up without disturbing the other project. The cluster account also runs a separate research project whose final evaluation depends on bit-exact caches. Everything here (two new Python environments, model downloads, Hugging Face / Triton / vLLM / pip caches) lives under `~/IE643_Project`. One leak was found and fixed: vLLM and flashinfer wrote caches into the shared home folder; they were moved and redirected.

2. Getting the software stack to run took several rounds:

- The GPU driver supports CUDA 12.8, but the newest vLLM needs a newer PyTorch with no CUDA-12.8 build → vLLM 0.26 with a CUDA 12.9 build (CUDA minor-version compatibility, verified on the GPU).
- `import vllm` failed three times in a row: a video-decoding library built for CUDA 13, then missing FFmpeg libraries, then a C++ runtime that was too old. Each was fixed inside our own environment.
- vLLM's default sampler tries to compile CUDA code at run time, and the cluster has no CUDA compiler → switched to vLLM's PyTorch sampler.
- transformers picks the fast Triton kernels for Qwen3.5 even on a CPU, where Triton crashes → a switch that disables them on CPU-only runs. Even then, **training Qwen3.5 on a CPU was hopeless** (one step of 870 tokens did not finish in 25 minutes), so all training happened on GPUs.
- The installed library versions had renamed training arguments; the 27B model needed fewer parallel sequences to fit next to its weights.

3. Verifying before spending GPU time. Before the real runs: all 28 unit tests (oracle = labels on all 5,000 samples, validator silent on all gold graphs, metrics behave on corruptions), a merge check (merged weights reproduce the adapter within bf16 rounding), a dry run of the whole evaluation chain on fake predictions, a GPU smoke test (9B: 12 s per step, 21 GB), and a token-level check that the model is trained on exactly the answer plus one end token.

4. The prompt bug. The first real zero-shot run scored **0%** valid: 957 of 1,000 answers failed only because the spec's prompt never said that the input node's `kind` is `"placeholder"` (the models wrote `"input"`). We clarified the rule, asked for compact one-line JSON (12% of answers had run out of tokens while pretty-printing), rebuilt the training files and restarted every job. Zero-shot rose to 19.9%; the remaining zero-shot failures are genuine (e.g. the runaway loop in Example 6).

5. Thinking mode needed room. With the spec's 8,192-token budget, 97% of thinking answers were still reasoning when cut off. With 30,000 tokens most finished, with excellent topology but missing shapes; forcing the JSON format *after* the reasoning ("think-then-constrain") fixed the format. Thinking costs 17–22k tokens per graph (about 20× a normal answer).

6. The night of experiments. After the planned runs, the weak spot was clear: models fine-tuned on 54 templates fail on new code. We added the synthetic-data arm, the 27B ceiling, combinations of retry and self-consistency, and OOD-2. Every change is recorded with its reason in `DECISIONS.md`; the full log is `PROJECT_LOG.md`.

5 Results

5.1 The main table

Configuration	**fully valid, %** test (1,000)	test_anon	(exact match % in brackets) novel_topo (564)	OOD-1 (40)	OOD-2 (21)
2B 3-shot	1.1 (6.5)	–	1.1 (3.2)	0.0 (0.0)	0.0 (4.8)
Coder-7B 3-shot	15.4 (21.7)	–	6.9 (19.1)	10.0 (5.0)	47.6 (33.3)
9B zero-shot	19.9 (20.3)	–	10.3 (14.0)	12.5 (5.0)	38.1 (28.6)
9B 3-shot	54.9 (57.1)	47.0 (50.4)	32.3 (33.9)	27.5 (30.0)	47.6 (38.1)
9B 3-shot + retry	–	–	–	45.0 (40.0)	71.4 (66.7)
27B zero-shot + retry	–	–	87.9 (90.8)	**100** (87.5)	90.5 (85.7)
27B 3-shot	91.0 (97.9)	–	92.4 (99.1)	80.0 (70.0)	**95.2** (90.5)
2B fine-tuned	93.3 (93.7)	93.4	64.4 (64.5)	5.0 (2.5)	28.6 (19.0)
2B FT + synthetic	**100** (100)	99.8	**100** (100)	72.5 (70.0)	71.4 (52.4)
9B fine-tuned	99.7 (99.7)	99.6	**100** (100)	47.5 (30.0)	85.7 (71.4)
9B fine-tuned + retry	99.7 (99.7)	–	**100** (100)	67.5 (45.0)	90.5 (76.2)
9B FT + synthetic	**100** (100)	**100**	**100** (100)	95.0 (97.5)	76.2 (76.2)
9B FT + synthetic + retry	**100** (100)	–	**100** (100)	97.5 (97.5)	90.5 (76.2)

Table 1: Headline results (all constrained decoding, greedy). "–" = not run. "Fully valid" = passes all validator levels; "exact" = identical nodes and edges (ignores shapes, so it can exceed validity). On the small OOD sets one module is 2.5 (OOD-1) or 4.8 (OOD-2) percentage points.

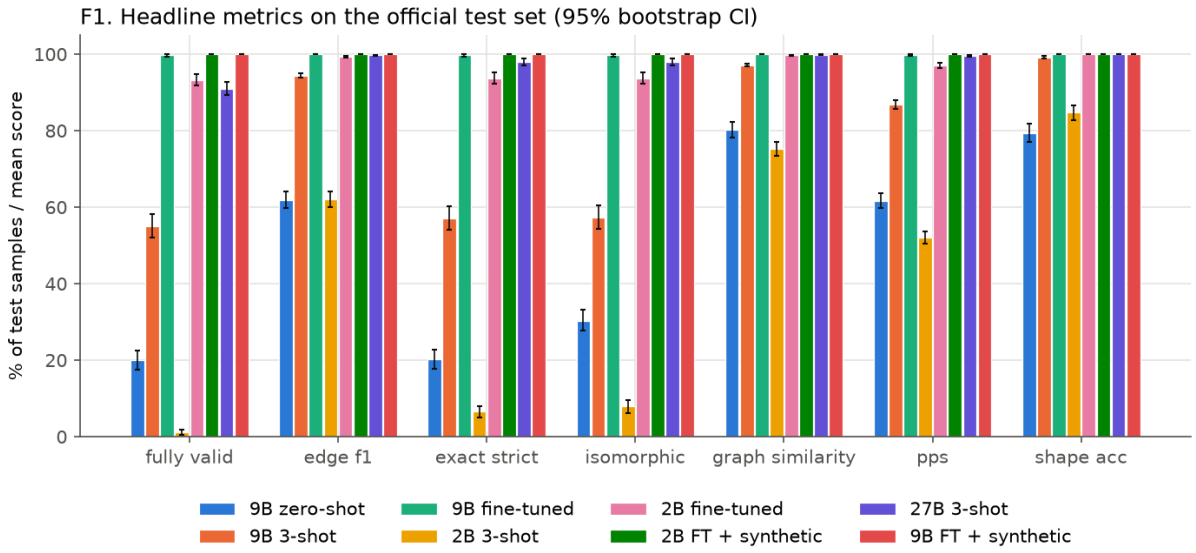


Figure 4: Headline metrics on the official test with 95% bootstrap confidence intervals (F1).

5.2 Complexity: which architectures are hard?

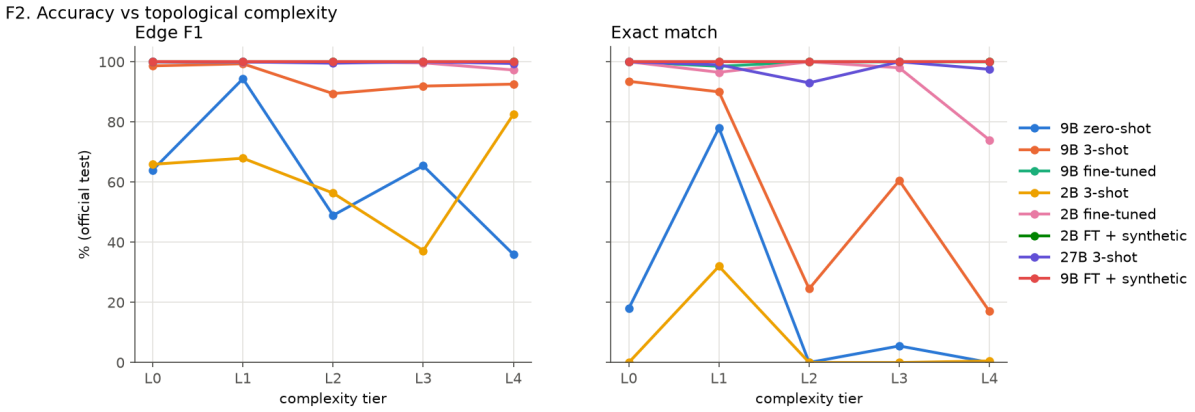


Figure 5: Edge F1 and exact match by tier (official test). For the prompted models, L2 (residual) and L4 (nested) are the hard tiers: 9B 3-shot exact match is 93.5% on L0 but 24.5% on L2 and 17.0% on L4. The long skip edge into an `add` is the typical miss. Fine-tuned models are flat at ≈100%.

5.3 Memorisation versus generalisation

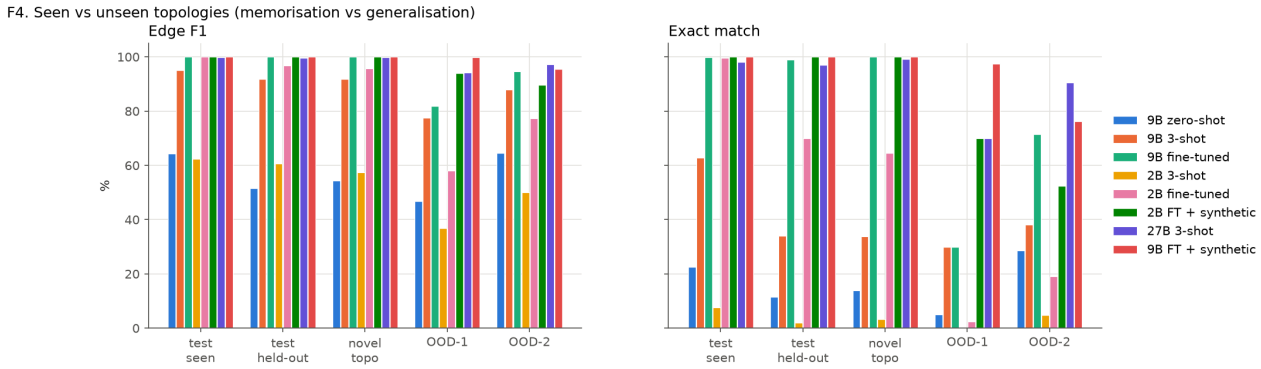


Figure 6: From left to right the test sets move further from the training data: test samples with a seen topology, test samples with a held-out topology, the novel_topo set, OOD-1 and OOD-2. Plain fine-tuning (light green, pink) is near-perfect until OOD-1, where it collapses; the synthetic-data models (dark green, red) and the 27B (purple) hold up.

Inside the test set, the fine-tuned 2B scores 99.3% on seen topologies but 69.5% on held-out ones, and **0% on the held-out L4 topology**; the fine-tuned 9B scores 99.0% on held-out topologies including 100% on that L4 structure (Example 5). Scale decides whether a fine-tuned model memorises templates or learns the rules.

5.4 Robustness, validity and self-correction

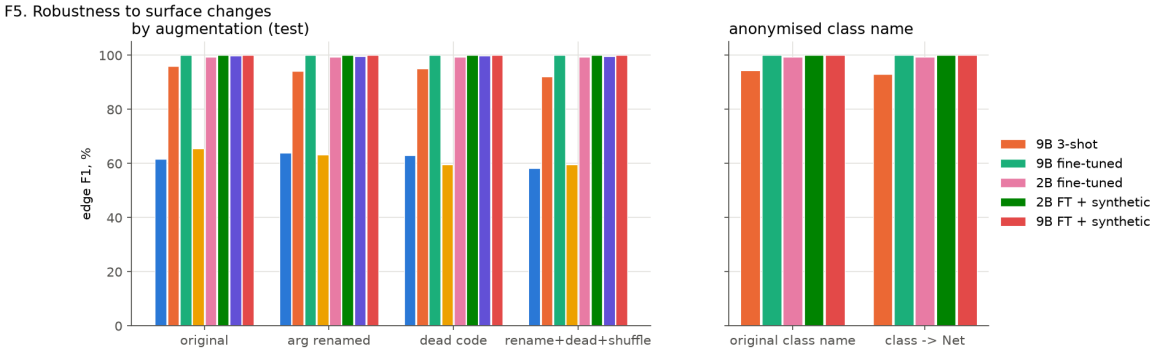


Figure 7: Robustness to renaming, dead code and anonymised class names (F5). The 9B 3-shot model drops from 54.9% to 47.0% valid when class names are replaced by `Net` (it uses the tier hint in the name); fine-tuned models do not care (93.3/93.4% for 2B, 99.7/99.6% for 9B).

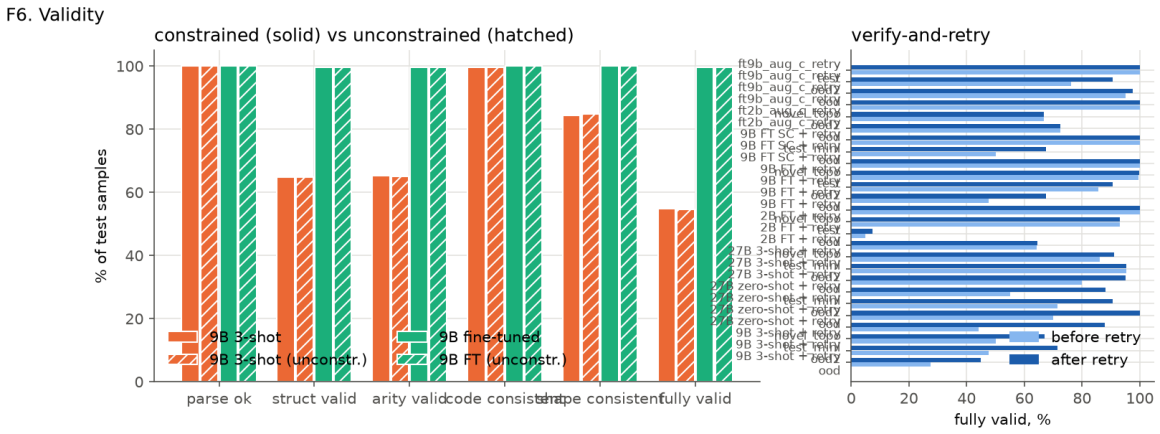


Figure 8: Left: validity levels with (solid) and without (hatched) constrained decoding. Constraining the JSON hardly matters (9B 3-shot: 54.9% vs 54.6%; fine-tuned 9B: identical), because these models already write valid JSON; the errors are in the content. Right: verify-and-retry before and after (F6).

Model	set	valid before retry	valid after retry	mean rounds
27B zero-shot	OOD-1	70.0	**100.0**	0.33
27B zero-shot	novel_topo	44.1	87.9	0.79
9B fine-tuned	OOD-1	47.5	67.5	0.93
9B FT + synthetic	OOD-2	76.2	90.5	0.38
9B 3-shot	OOD-1	27.5	45.0	1.28
2B fine-tuned	OOD-1	5.0	7.5	1.90
2B FT + synthetic	OOD-1	72.5	72.5	0.55

Table 2: Verify-and-retry uses only the validator, never the answer. It helps the 9B and 27B models a lot and the 2B models not at all: small models cannot act on the feedback.

5.5 Failure modes, properties, metric check, training

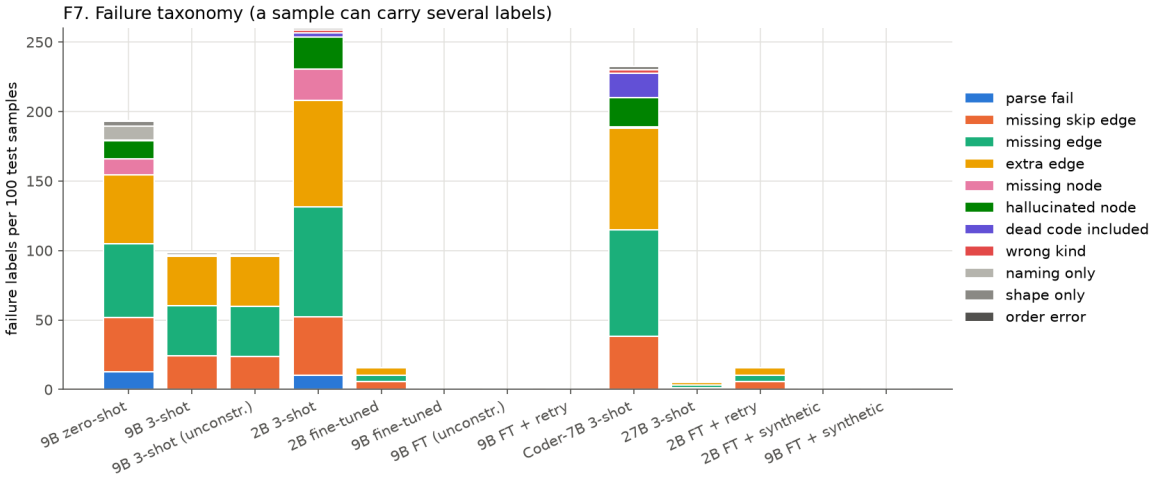


Figure 9: Failure labels per 100 test samples (a sample can have several). Prompted models mostly miss edges (ordinary and skip edges) and invent extra ones; parse failures are the zero-shot runaway loops.

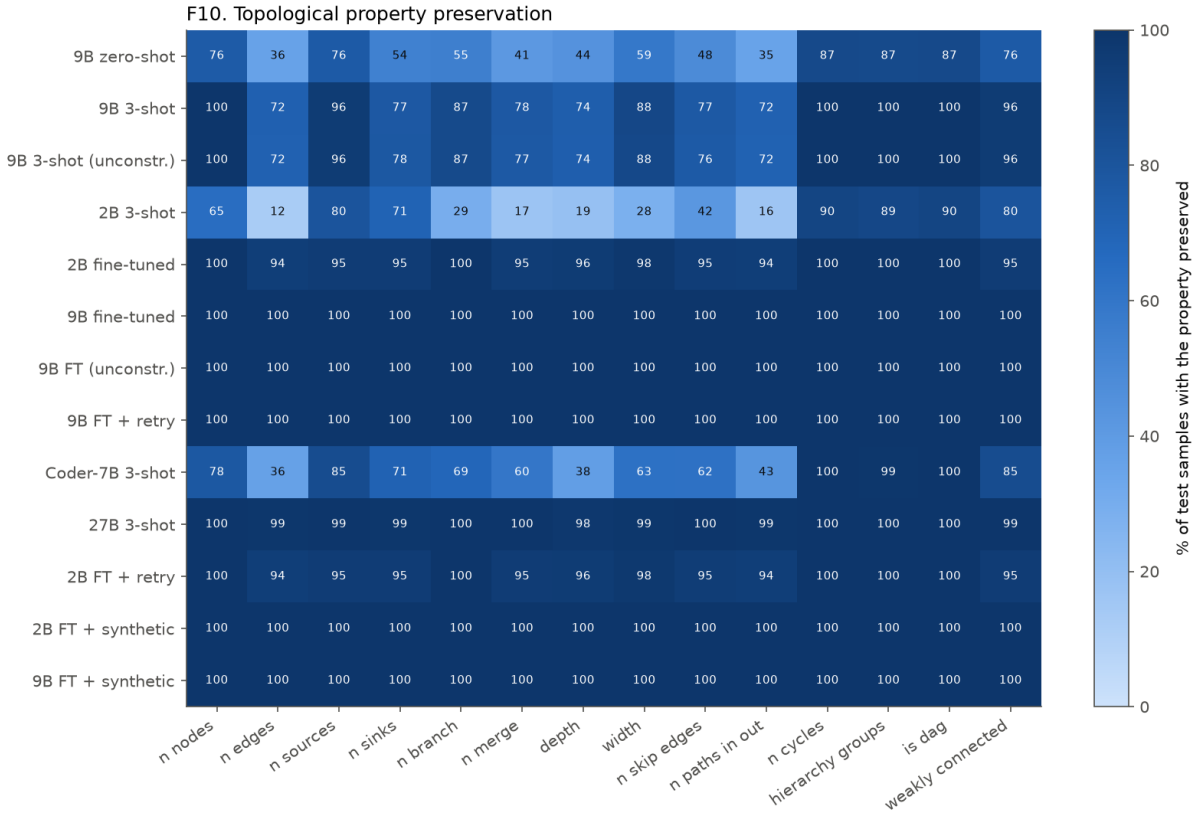


Figure 10: How often each topological property of the reference is preserved (official test).

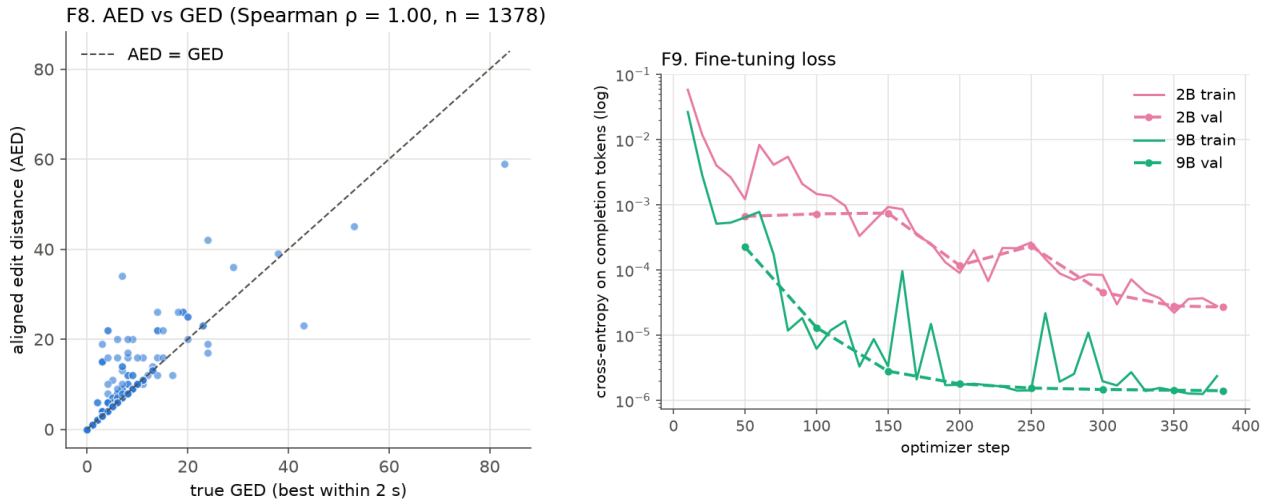


Figure 11: Left (F8): our fast aligned edit distance vs the true graph edit distance (Spearman 0.998; it is never below the true value). Right (F9): training and validation loss of the 2B and 9B fine-tunes (log scale); both reach losses near zero, as expected on templated data.

6 Twelve worked examples, from code to graph

Each example shows the code, a small results table, and drawings: the reference (from `torch.fx`) and one or two model predictions, usually as a **diff** against the reference. In a diff, black edges are correct, red dashed nodes and edges are in the reference but missing from the prediction, orange ones are invented by the model, and red text marks a wrong layer type or shape.

6.1 Example 1: A neuron-level MLP (tier L0)

set: official test • sample: `L0_0803` • tier L0 • augmentation: `aug_variant_3` • topology seen in training • 10 nodes

```
class TierL0_ToyMLP_201(nn.Module):
    def __init__(self):
        super().__init__()
        self.out = nn.Linear(3, 1)
        self.fc1_u2 = nn.Linear(2, 1)
        self.unused_op_35 = nn.Identity()
        self.fc1_u0 = nn.Linear(2, 1)
        self.fc1_u1 = nn.Linear(2, 1)

    def forward(self, inp_tensor):
        u0 = torch.relu(self.fc1_u0(inp_tensor))
        u1 = torch.relu(self.fc1_u1(inp_tensor))
        u2 = torch.relu(self.fc1_u2(inp_tensor))
        h = torch.cat([u0, u1, u2], dim=1)
        y = self.out(h)
        _dead_val = self.unused_op_35(inp_tensor)
        return y
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B zero-shot	no	0.55	no	0.88	268	naming only
9B fine-tuned	yes	1.00	yes	1.00	263	–

What to look at. Every hidden unit is its own `Linear(k,1)` ("neurons" made explicit). This sample is an *augmented* variant: the forward argument is renamed (`inp_tensor`), a dead layer is inserted and the `__init__` order is shuffled. The zero-shot 9B model wires the graph correctly (it is *isomorphic* to the reference) but names the input after the argument instead of `x`, picks the wrong input width ($[1,3]$ instead of $[1,2]$) and therefore gets node and edge *names* wrong: strict edge F1 is only 0.55 although the shape of the graph is right. The fine-tuned 9B reproduces the reference exactly.

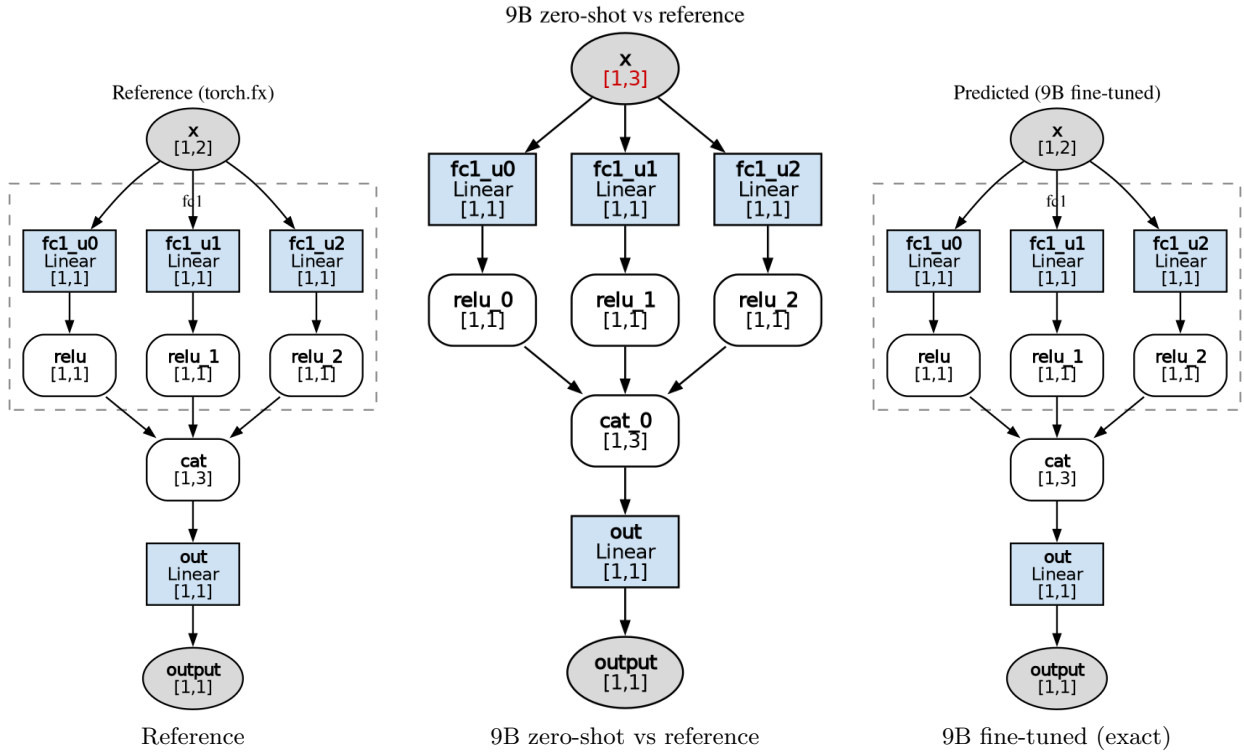


Figure 12: Example 1 (L0_0803). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.2 Example 2: A linear chain with dead code (tier L1)

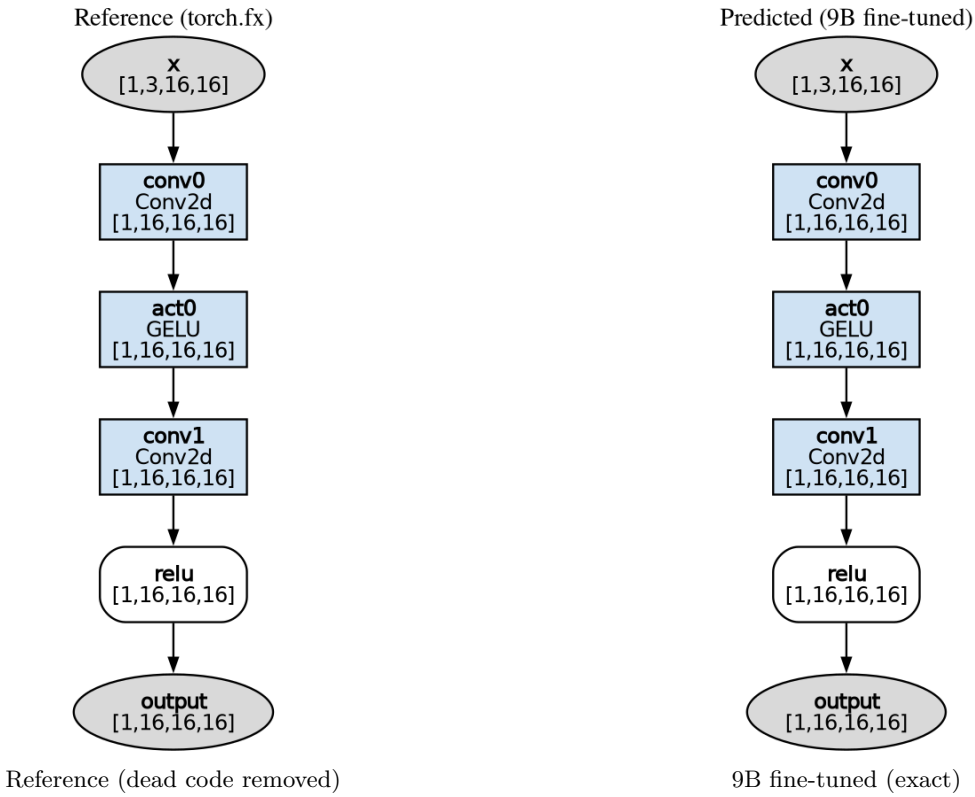
set: official test • sample: `L1_0886` • tier L1 • augmentation: `aug_variant_2` • topology seen in training • 6 nodes

```python
class TierL1_LinearChain_222(nn.Module):
    def __init__(self):
        super().__init__()
        self.unused_op_77 = nn.Identity()
        self.conv0 = nn.Conv2d(3, 16, kernel_size=5, padding=2)
        self.act0 = nn.GELU()
        self.conv1 = nn.Conv2d(16, 16, kernel_size=5, padding=2)

    def forward(self, x):
        x = self.conv0(x)
        x = self.act0(x)
        x = self.conv1(x)
        x = torch.relu(x)
        _dead_val = self.unused_op_77(x)
        return x
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B fine-tuned	yes	1.00	yes	1.00	196	–

What to look at. `self.unused_op_*` is declared and even *called*, but its result never reaches the `return`, so `torch.fx` (after dead-code elimination) does not contain it. The validator checks exactly this (rule "dead code included"). The fine-tuned model skips the dead call correctly. Over the whole test set the base Coder-7B model included dead code in 34.6% of the dead-code samples, the 9B 3-shot model in 0%.



Reference (dead code removed) 9B fine-tuned (exact)

Figure 13: Example 2 (L1_0886). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.3 Example 3: A residual network (tier L2): the missing skip edge

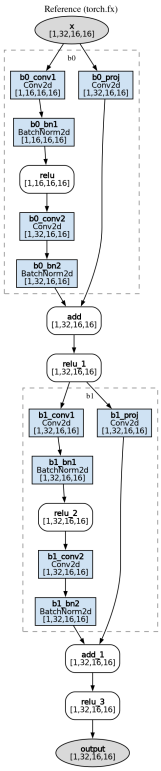
set: official test • sample: `L2_0804` • tier L2 • topology **held out** from training • 18 nodes

```
class TierL2_Residual_202(nn.Module):
    def __init__(self):
        super().__init__()
        self.b0_conv1 = nn.Conv2d(32, 16, kernel_size=5, padding=2)
        self.b0_bn1 = nn.BatchNorm2d(16)
        self.b0_conv2 = nn.Conv2d(16, 32, kernel_size=1)
        self.b0_bn2 = nn.BatchNorm2d(32)
        self.b0_proj = nn.Conv2d(32, 32, kernel_size=1)
        self.b1_conv1 = nn.Conv2d(32, 32, kernel_size=5, padding=2)
        self.b1_bn1 = nn.BatchNorm2d(32)
        self.b1_conv2 = nn.Conv2d(32, 32, kernel_size=1)
        self.b1_bn2 = nn.BatchNorm2d(32)
        self.b1_proj = nn.Conv2d(32, 32, kernel_size=1)

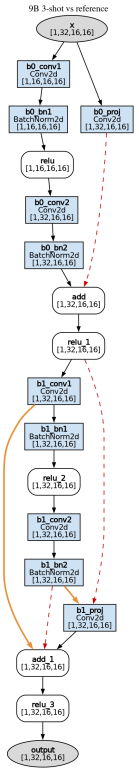
    def forward(self, x):
        res_0 = x
        out_0 = torch.relu(self.b0_bn1(self.b0_conv1(x)))
        out_0 = self.b0_bn2(self.b0_conv2(out_0))
        res_0 = self.b0_proj(res_0)
        x = torch.relu(out_0 + res_0)
        res_1 = x
        out_1 = torch.relu(self.b1_bn1(self.b1_conv1(x)))
        out_1 = self.b1_bn2(self.b1_conv2(out_1))
        res_1 = self.b1_proj(res_1)
        x = torch.relu(out_1 + res_1)
        return x
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B 3-shot	no	0.86	no	1.00	706	missing skip edge, missing edge, extra edge
9B fine-tuned	yes	1.00	yes	1.00	644	–

What to look at. A residual network whose structure was *held out* from fine-tuning. The 3-shot base model gets every node right (node F1 1.0) but miswires the edges: the red dashed edges are reference edges it missed (including the long *skip connection* into an `add`), orange edges are ones it invented, and it emits edges to node indices that do not exist (`edge [17, 18] points outside the 18 nodes`). Long-range skip edges are the single most common failure of the prompted models (the `missing_skip_edge` label in Figure 9). The fine-tuned 9B gets this held-out topology exactly right.



Reference



9B 3-shot vs reference

Figure 14: Example 3 (L2_0804). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.4 Example 4: Parallel branches joined by concat (tier L3)

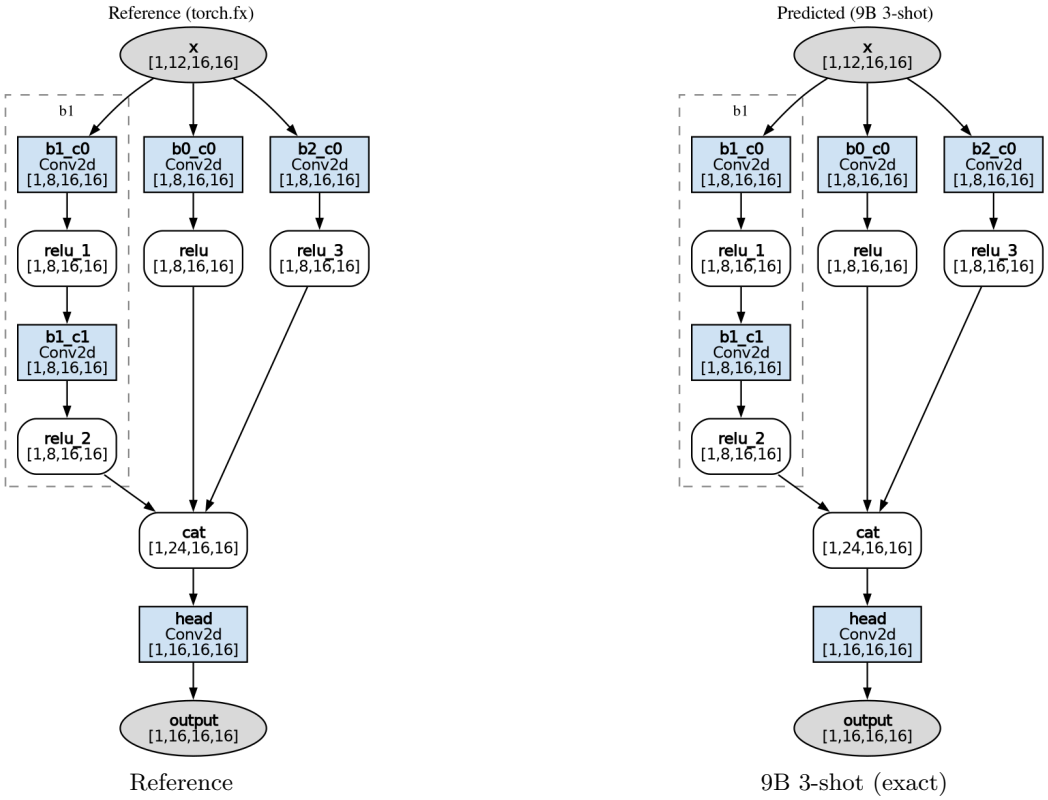
set: official test • sample: `L3_0804` • tier L3 • topology seen in training • 12 nodes

```
class TierL3_MultiBranch_202(nn.Module):
    def __init__(self):
        super().__init__()
        self.b0_c0 = nn.Conv2d(12, 8, kernel_size=5, padding=2)
        self.b1_c0 = nn.Conv2d(12, 8, kernel_size=1, padding=0)
        self.b1_c1 = nn.Conv2d(8, 8, kernel_size=1)
        self.b2_c0 = nn.Conv2d(12, 8, kernel_size=3, padding=1)
        self.head = nn.Conv2d(24, 16, kernel_size=1)

    def forward(self, x):
        b0 = torch.relu(self.b0_c0(x))
        b1 = torch.relu(self.b1_c0(x))
        b1 = torch.relu(self.b1_c1(b1))
        b2 = torch.relu(self.b2_c0(x))
        merged = torch.cat([b0, b1, b2], dim=1)
        out = self.head(merged)
        return out
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B 3-shot	yes	1.00	yes	1.00	405	–

What to look at. Several parallel branches read the same tensor and are joined by `torch.cat`. Branching and merging nodes are easy to spot in the drawing (out-degree > 1, in-degree > 1). Here even the prompted 3-shot model is exactly right; tier L3 is the easiest multi-path tier for it (60.5% exact).



Reference | 9B 3-shot (exact)

Figure 15: Example 4 (L3_0804). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.5 Example 5: A held-out nested topology (tier L4): memorisation vs generalisation

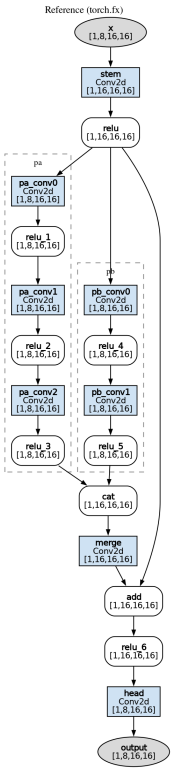
set: official test • sample: `L4_0800` • tier L4 • topology **held out** from training • 19 nodes

```
class TierL4_Nested_201(nn.Module):
    def __init__(self):
        super().__init__()
        self.stem = nn.Conv2d(8, 16, kernel_size=3, padding=1)
        self.pa_conv0 = nn.Conv2d(16, 8, kernel_size=1, padding=0)
        self.pa_conv1 = nn.Conv2d(8, 8, kernel_size=1, padding=0)
        self.pa_conv2 = nn.Conv2d(8, 8, kernel_size=3, padding=1)
        self.pb_conv0 = nn.Conv2d(16, 8, kernel_size=3, padding=1)
        self.pb_conv1 = nn.Conv2d(8, 8, kernel_size=1, padding=0)
        self.merge = nn.Conv2d(16, 16, kernel_size=1)
        self.head = nn.Conv2d(16, 8, kernel_size=1)

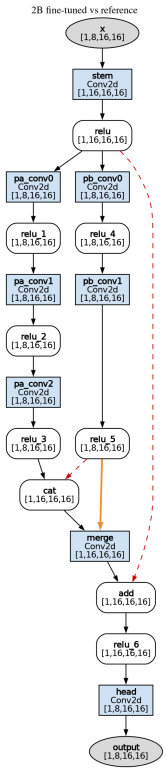
    def forward(self, x):
        stem_out = torch.relu(self.stem(x))
        pa = stem_out
        pb = stem_out
        pa = torch.relu(self.pa_conv0(pa))
        pa = torch.relu(self.pa_conv1(pa))
        pa = torch.relu(self.pa_conv2(pa))
        pb = torch.relu(self.pb_conv0(pb))
        pb = torch.relu(self.pb_conv1(pb))
        cat_ab = torch.cat([pa, pb], dim=1)
        merged = self.merge(cat_ab)
        combined = torch.relu(merged + stem_out)
        out = self.head(combined)
        return out
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
2B fine-tuned	no	0.92	no	1.00	638	missing skip edge, extra edge
9B fine-tuned	yes	1.00	yes	1.00	643	–

What to look at. The nested L4 pattern (stem → two paths → concat → merge → residual add → head) with the one L4 structure that was held out of training. The fine-tuned **2B** writes the nodes correctly but wires them like the L4 templates it saw in training: the validator reports `'cat' has 1 input`, `'merge' has 2 inputs; a Conv2d takes exactly 1` and `'add' has 1 input (missing skip edge?)`. All 52 held-out L4 test samples fail this way for the 2B (0%), while the fine-tuned **9B** gets 100% of them. This is the clearest memorisation-vs-generalisation contrast in the project.



Reference



2B fine-tuned vs reference

Figure 16: Example 5 (L4_0800). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.6 Example 6: The zero-shot runaway loop

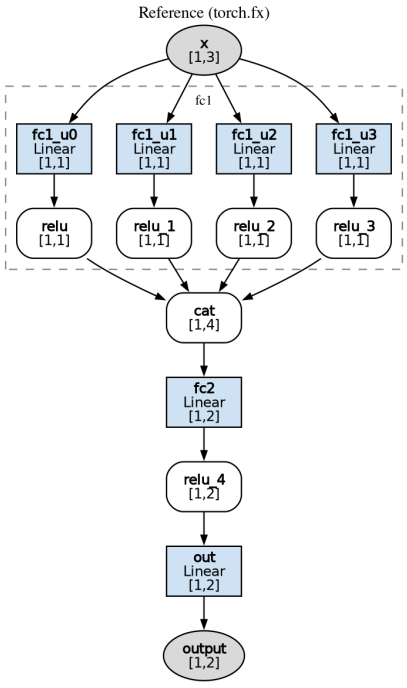
set: official test • sample: `L0_0857` • tier L0 • augmentation: `aug_variant_1` • topology seen in training • 14 nodes

```
class TierL0_ToyMLP_215(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1_u0 = nn.Linear(3, 1)
        self.fc1_u1 = nn.Linear(3, 1)
        self.fc1_u2 = nn.Linear(3, 1)
        self.fc1_u3 = nn.Linear(3, 1)
        self.fc2 = nn.Linear(4, 2)
        self.out = nn.Linear(2, 2)

    def forward(self, inp_tensor):
        u0 = torch.relu(self.fc1_u0(inp_tensor))
        u1 = torch.relu(self.fc1_u1(inp_tensor))
        u2 = torch.relu(self.fc1_u2(inp_tensor))
        u3 = torch.relu(self.fc1_u3(inp_tensor))
        h = torch.cat([u0, u1, u2, u3], dim=1)
        h = torch.relu(self.fc2(h))
        y = self.out(h)
        return y
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B zero-shot	no	0.00	no	0.00	3072	parse fail

What to look at. Nothing is wrong with the code; the failure is in decoding. The zero-shot 9B starts a plausible answer, then falls into a loop in the edge list (`[n,n+1]` with ever larger `n`) until the 3,072-token limit stops it. The output is not a usable graph (parse/schema failure). 22% of zero-shot test answers ended like this (mostly on the larger L2/L4 graphs); fine-tuned models never did (Section 4).



Reference

Figure 17: Example 6 (L0_0857). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

End of the zero-shot answer (cut off at 3,072 tokens):

```
..., 354], [354, 355], [355, 356], [356, 357], [357, 358], [358, 359], [359, 360], [360, 361], [361, 362], [362,
      363], [363, 364], [364, 365], [365, 366], [366, 367], [367, 368], [368, 369], [369, 370], [370, 371],
     [371, 372], [372,
```

6.7 Example 7: OOD: nested submodules (dotted scopes)

set: OOD-1 • sample: `OOD_resnet_nested_2` • category: `nested_basicblock` • 13 nodes

```
class ResUnit(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.conv_a = nn.Conv2d(c, c, 3, padding=1)
        self.conv_b = nn.Conv2d(c, c, 3, padding=1)

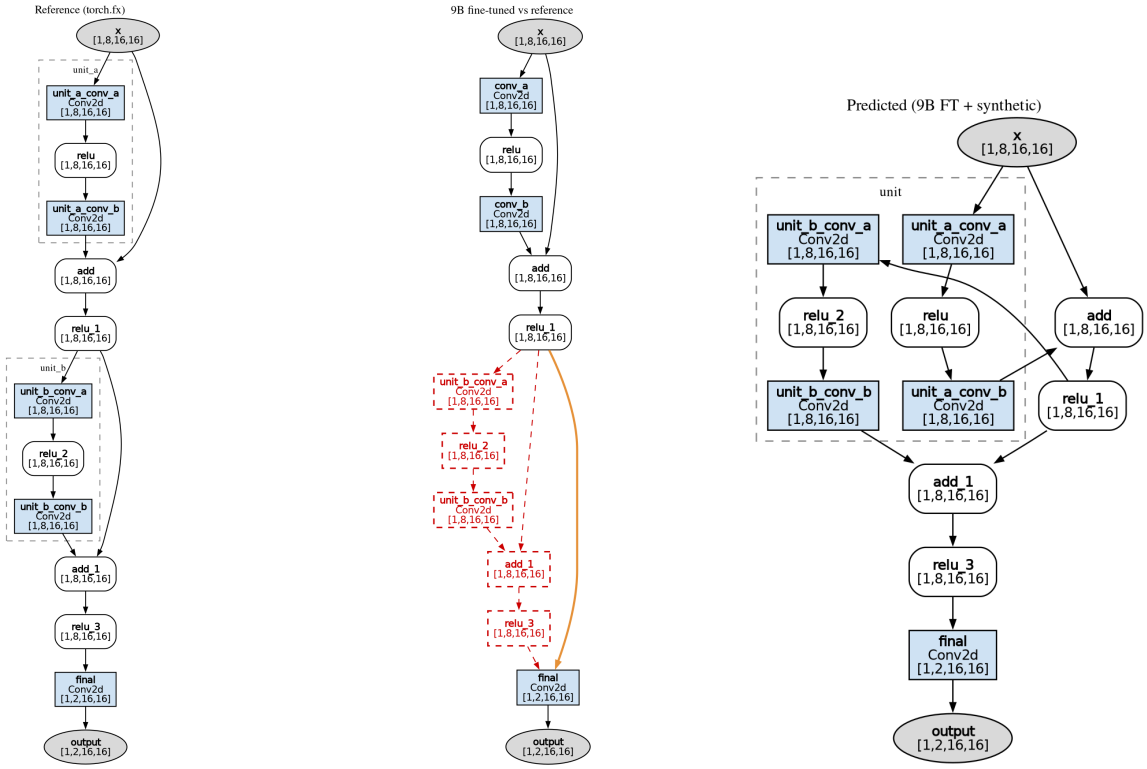
    def forward(self, inp):
        h = F.relu(self.conv_a(inp))
        return F.relu(self.conv_b(h) + inp)


class TwoUnitNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.unit_a = ResUnit(8)
        self.unit_b = ResUnit(8)
        self.final = nn.Conv2d(8, 2, 1)

    def forward(self, feat):
        return self.final(self.unit_b(self.unit_a(feat)))
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B fine-tuned	no	0.27	no	1.00	256	missing skip edge, missing edge, extra edge, missing node
9B FT + synthetic	yes	1.00	yes	1.00	435	–

What to look at. An out-of-distribution module: a *submodule class* `ResUnit` used twice. `torch.fx` inlines nested modules and names their layers with the full path (`unit_a_conv_a`, scope `unit_a.conv_a`). The dataset only has flat models, so the fine-tuned 9B writes `conv_a` (a name that is not declared at the top level: the validator flags it as possibly hallucinated) and collapses the two units into one (the whole second unit is red dashed = missing). After adding synthetic training modules that contain nested classes, the model gets it exactly right.



Reference | 9B fine-tuned vs reference | 9B FT + synthetic (exact)

Figure 18: Example 7 (OOD_resnet_nested_2). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.8 Example 8: OOD: an nn.ModuleList loop

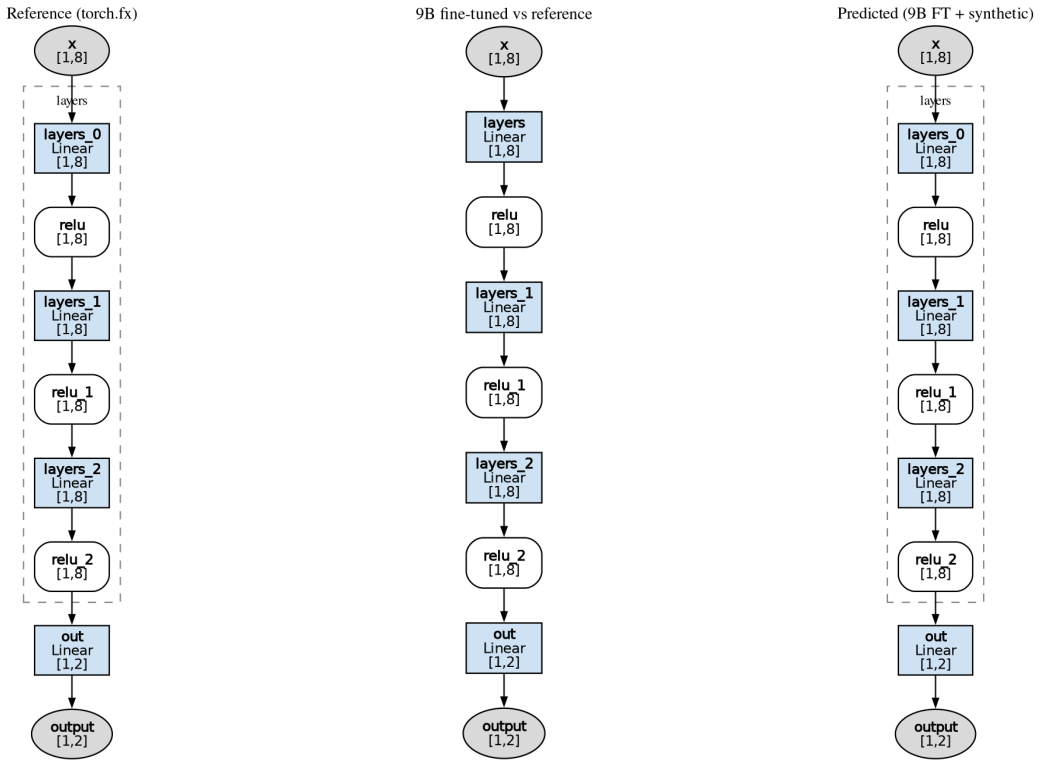
set: OOD-1 • sample: `OOD_modulelist_1` • category: `modulelist` • 9 nodes

```
class DeepMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.ModuleList([nn.Linear(8, 8) for _ in range(3)])
        self.out = nn.Linear(8, 2)

    def forward(self, x):
        for layer in self.layers:
            x = torch.relu(layer(x))
        return self.out(x)
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B fine-tuned	no	0.75	no	1.00	226	naming only
9B FT + synthetic	yes	1.00	yes	1.00	228	–

What to look at. `nn.ModuleList` plus a Python `for` loop: `torch.fx` unrolls the loop and names the layers `layers_0`, `layers_1`, `layers_2`. The fine-tuned 9B draws the right structure (isomorphic) but names the nodes `layers`, `layers_1`, …, so strict metrics and the code-consistency check fail ("naming only" failure). The synthetic-data model names them correctly.



Reference | 9B fine-tuned vs reference | 9B FT + synthetic (exact)

Figure 19: Example 8 (OOD_modulelist_1). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.9 Example 9: OOD: Squeeze-and-Excitation, fixed by verify-and-retry

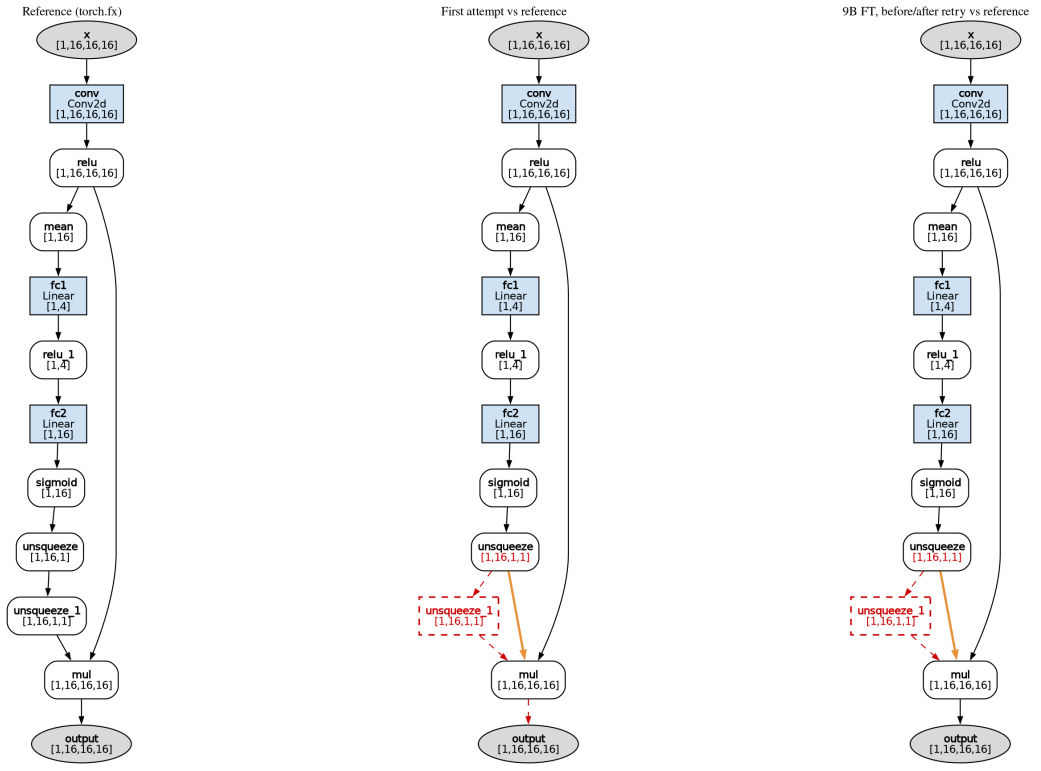
set: OOD-1 • sample: `OOD_se_1` • category: `squeeze_excite` • 12 nodes

```
class SEBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(16, 16, 3, padding=1)
        self.fc1 = nn.Linear(16, 4)
        self.fc2 = nn.Linear(4, 16)

    def forward(self, x):
        y = torch.relu(self.conv(x))
        s = y.mean((2, 3))
        s = torch.sigmoid(self.fc2(torch.relu(self.fc1(s))))
        s = s.unsqueeze(-1).unsqueeze(-1)
        return y * s
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B FT, before/after retry	yes	0.87	no	0.91	635	missing edge, extra edge, missing node

What to look at. *Verify-and-retry*: the validator's messages (which never use the reference) are sent back to the model as a new chat turn. The first attempt left the output node disconnected (`output node needs >=1 input`, `node 0 'x' does not lead to the output`). After one retry round the graph passes every validity check, but it is *still not exactly right* (edge F1 0.87, one node missing): a valid graph is not necessarily a correct graph. Over OOD-1, retry raised the fine-tuned 9B from 47.5% to 67.5% valid and from 30% to 45% exact.



Reference | first attempt vs reference | after 1 retry round

Figure 20: Example 9 (OOD_se_1). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.10 Example 10: OOD-2: a transposed-convolution decoder (shape failure)

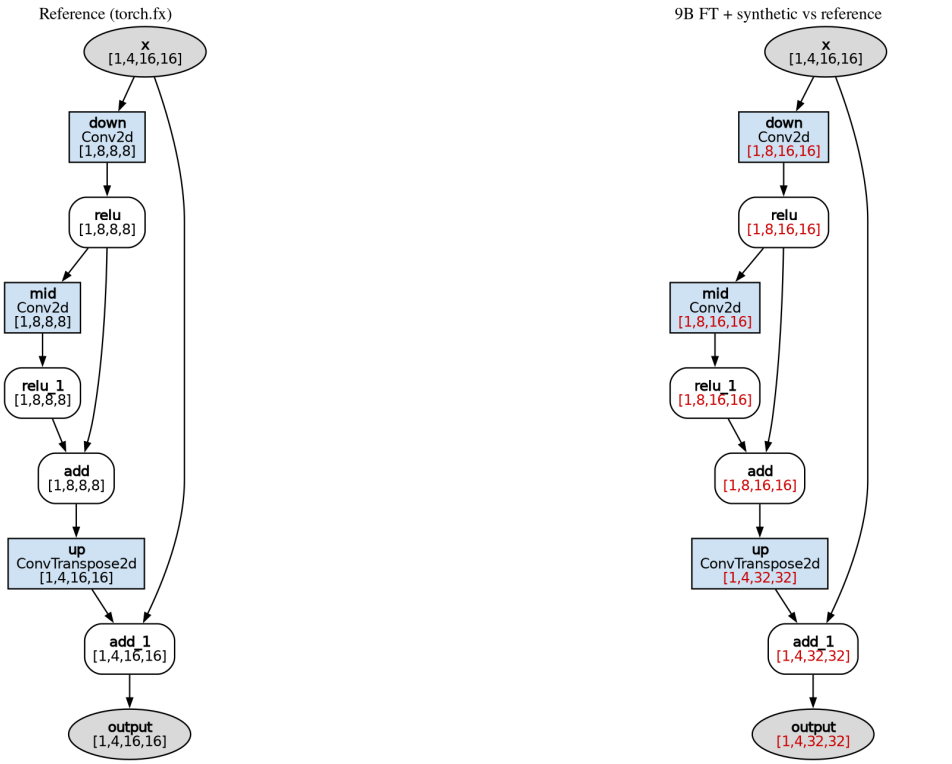
set: OOD-2 • sample: `OOD2_deconv_2` • category: `transposed_conv` • 9 nodes

```
class DeconvSkip(nn.Module):
    def __init__(self):
        super().__init__()
        self.down = nn.Conv2d(4, 8, 3, stride=2, padding=1)
        self.mid = nn.Conv2d(8, 8, 3, padding=1)
        self.up = nn.ConvTranspose2d(8, 4, 2, stride=2)

    def forward(self, x):
        d = F.relu(self.down(x))
        m = F.relu(self.mid(d)) + d
        return self.up(m) + x
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B FT + synthetic	no	1.00	yes	0.11	291	shape only
27B 3-shot	yes	1.00	yes	1.00	281	–

What to look at. OOD-2 deliberately uses operators that appear in neither the dataset nor the synthetic generator, here `nn.ConvTranspose2d` (an up-sampling convolution). The synthetic-data 9B gets the *topology* exactly right (all edges black) but almost all *shapes* wrong (red shape text, shape accuracy 11%). The mistake starts at the very first layer: the model ignores the `stride=2` of the ordinary convolution `down` and predicts `[1,8,16,16]` instead of `[1,8,8,8]`. Every later shape inherits the error, so the transposed convolution then "doubles" 16 to 32 instead of 8 to 16. The shape check catches the inconsistency at the final skip addition (`add_1 inputs [1,4,16,16] and [1,4,32,32] are not broadcast-compatible`). The 27B model gets this one fully right. Shape propagation through unseen operators is the main remaining weakness of all fine-tuned models.



Reference 9B FT + synthetic vs reference

Figure 21: Example 10 (OOD2_deconv_2). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.11 Example 11: OOD-2: attention with matmul and softmax

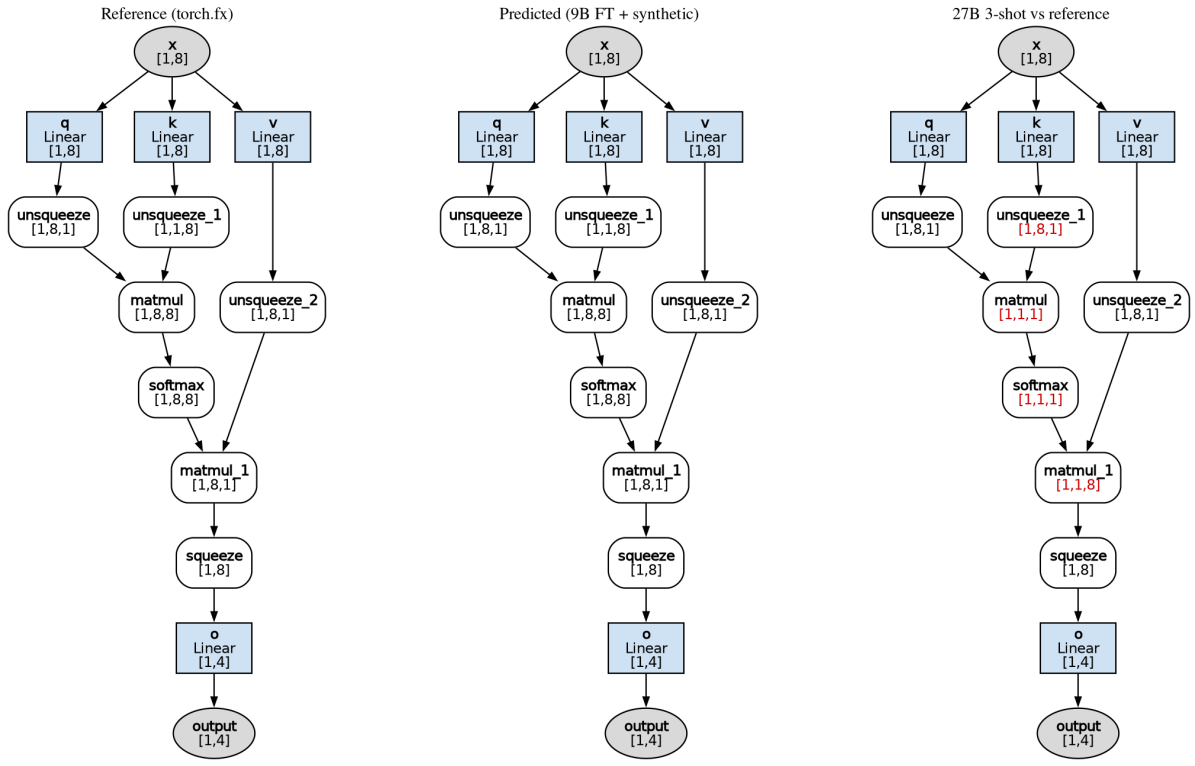
set: OOD-2 • sample: `OOD2_attn_1` • category: `matmul_attention` • 13 nodes

```python
class TinyAttention(nn.Module):
    def __init__(self):
        super().__init__()
        self.q = nn.Linear(8, 8)
        self.k = nn.Linear(8, 8)
        self.v = nn.Linear(8, 8)
        self.o = nn.Linear(8, 4)

    def forward(self, x):
        q = self.q(x).unsqueeze(2)
        k = self.k(x).unsqueeze(1)
        w = torch.softmax(torch.matmul(q, k), dim=-1)
        y = torch.matmul(w, self.v(x).unsqueeze(2)).squeeze(2)
        return self.o(y)
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B FT + synthetic	yes	1.00	yes	1.00	349	–
27B 3-shot	no	1.00	yes	0.69	355	shape only

What to look at. A tiny attention block (`unsqueeze`, `matmul`, `softmax`, `squeeze`; method calls become `call_method` nodes). Here the roles flip: the synthetic-data 9B is exactly right, while the 27B gets the topology right but adds an edge to a node index that does not exist and gets 31% of the shapes wrong, so the validator rejects it. Bigger is better on average, not on every example.



Reference | 9B FT + synthetic (exact) | 27B 3-shot vs reference

Figure 22: Example 11 (OOD2_attn_1). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

6.12 Example 12: Untraceable code: no torch.fx reference exists

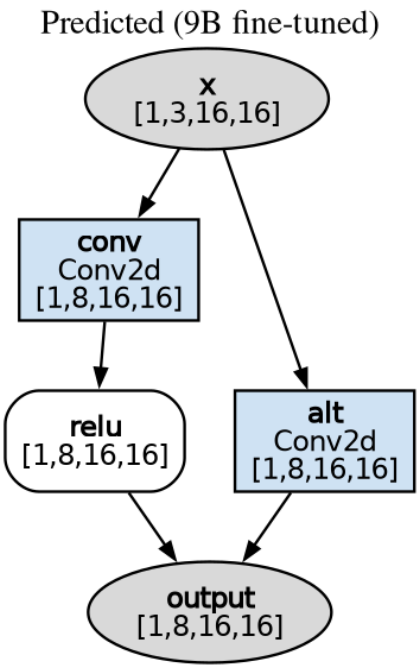
set: untraceable set • sample: `UNT_data_dependent_if` • category: `untraceable`

```
class GatedByMean(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(3, 8, 3, padding=1)
        self.alt = nn.Conv2d(3, 8, 1)

    def forward(self, x):
        if x.mean() > 0:
            return torch.relu(self.conv(x))
        return self.alt(x)
```

model	valid	edge F1	exact	shape acc.	tokens	failure labels
9B fine-tuned	no	–	n/a	–	160	n/a

What to look at. The `if` depends on the *value* of the input, so `torch.fx.symbolic_trace` raises `TraceError`: there is no reference graph, and this is exactly where an LLM can still be useful. The fine-tuned 9B draws both branches feeding the output (it cannot know which branch runs), which the validator reports as `output node has 2 inputs`. The validator also wrongly reports `conv` as dead code: its code analysis only follows straight-line code and does not look inside `if` bodies (a known limitation, Section 8). These examples are qualitative only.



9B fine-tuned prediction (no reference exists)

Figure 23: Example 12 (UNT_data_dependent_if). Diff colours: black = matched, red dashed = missing (reference only), orange = hallucinated (prediction only), red text = wrong kind or shape.

7 Findings

1. **Fine-tuning solves the dataset's own distribution.** The fine-tuned 9B is 99.7% valid on the official test (54.9% for the same model with 3 examples), ignores class names (99.6% anonymised) and is 100% on topologies it never saw (`novel_topo`). Even the 2B reaches 93.3%.
2. **Whether fine-tuning memorises depends on scale.** The fine-tuned 2B fails every held-out L4 graph (0/52) by reproducing training templates; the 9B gets all of them.
3. **Plain fine-tuning breaks on genuinely new code.** On OOD-1 the fine-tuned models fall to 5% (2B) and 47.5% (9B): they have never seen nested submodules, `ModuleList` naming or pooling (Examples 7, 8).
4. **Cheap synthetic data fixes most of it.** Adding ∼2,800 random, automatically labelled modules lifts OOD-1 to 95% (9B) and 72.5% (2B), and the 2B reaches 100% on test and novel_topo. On OOD-2 (operators outside both training sources) the synthetic-data models get the *structure* better than before but still miss *shapes* of unseen operators (Example 10).
5. **Scale + feedback is the most robust option for unfamiliar code.** The 27B model without any fine-tuning reaches 100% valid on OOD-1 with verify-and-retry and 95.2% on OOD-2 with 3 examples.
6. **Verify-and-retry works for large models only** (+20 points for the fine-tuned 9B on OOD-1, 70%→100% for the 27B; nothing for 2B models). Valid after retry does not mean correct (Example 9).
7. **Smaller findings.** Constrained decoding barely matters for these models. Long skip edges are the hardest structure for prompted models. The older code-specialised Qwen2.5-Coder-7B is far behind the general Qwen3.5-9B (15.4% vs 54.9% at 3-shot) and includes dead code in a third of the dead-code samples. Thinking mode gives excellent topology when it finishes but costs ∼20× the tokens.

8 Caveats and limitations

- **Small OOD sets.** OOD-1 has 40 modules and OOD-2 has 21; one module moves a score by 2.5 or 4.8 points. Differences of one or two modules (e.g. 27B 3-shot 80% vs synthetic-data 2B 72.5% on OOD-1) are not significant.
- **The same author wrote OOD-1 and the synthetic generator**, so they share an operator vocabulary. OOD-1 gains of the synthetic-data models partly measure "familiar operators, new code". OOD-2 was written afterwards to test beyond that vocabulary, but it was also written by us, not by independent people.
- **Dataset conventions are narrow**: batch 1, 16×16 inputs, mostly flat modules, 54 topologies in total. The models learned these conventions; real-world code (other input sizes, huge models) is out of scope.
- **The validator is approximate for control flow**: it reads straight-line code and does not look inside `if` or `while` bodies (Example 12 shows a false "dead code" message). It is exact on all dataset and OOD graphs, which are straight-line.
- **Exact match ignores invalid edges**: an edge pointing to a non-existent node is dropped by the metric but rejected by the validator, so a graph can be "exact" but not "valid" (Example 11). We always report both.
- **Tuple outputs are not supported** by the output format (no tensor shape), so three OOD-2 modules using `torch.chunk/split` were dropped.
- **Single runs**: every configuration was run once with greedy decoding; the confidence intervals reflect sample variation, not training-seed variation. All inference ran on one GPU type (L40S).

- **Not run**: the planned P2 ablations "train on chain-of-thought targets" and "edge-list output format". The 27B model was run in FP8 (the bf16 version does not fit on one 48 GB GPU).

9 Reproducing and exploring

```
source ~/IE643_Project/tgenv.sh train # or: infer (activates an env and sets every cache)
python scripts/prepare_data.py --tokenizer $MODELS/Qwen3.5-9B # splits, SFT files, token audit, stats
python scripts/build_ood.py; python scripts/build_ood.py ood2 # label the OOD sets
pytest -q # 28 tests
sbatch -p l40 --qos=l40 jobs/train.sh # LoRA 2B + 9B, merge
sbatch -p l40 --qos=l40 --export=ALL,CONFIGS="ft9b_c" jobs/infer.sh # any config in configs/experiments.yaml
bash jobs/cpu_eval.sh # metrics, tables, figures, gallery (CPU)
python app/app.py --mode precomputed # demo app without a GPU
```

Results live in `results/tables` (CSV + Markdown, including `main.md`), `results/figures` and `results/figures/gallery` (best/median/worst per tier and OOD examples). The reasons behind every deviation from the original build spec are in `DECISIONS.md`.

The appendices hold every remaining detail: the complete result tables (every configuration on every set, by tier, size, variant, topology and OOD category, validity levels, retry, properties, failure labels, the edit-distance check, token and speed statistics; Appendix B), the data in detail (Appendix C), the exact prompts and output schema (Appendix D), every validator check and metric definition (Appendix E), training and inference settings (Appendix F), software, hardware and the full compute log (Appendix G), the decision log, the problems and fixes, the timeline (Appendices H–J), a file-by-file inventory with the tests (Appendix K) and the demo app (Appendix L).

A Glossary

torch.fx
: PyTorch tool that records the operations a module performs; our source of exact answers ("oracle").

placeholder / call_module / call_function / call_method / output
: the node types of a torch.fx graph: input, a layer object (`self.conv`), a function (`torch.relu`, `a+b`), a tensor method (`x.mean`), and the returned value.

CG-JSON
: our compact output format: a node list and edges as index pairs.

Topology / structure
: the graph with names removed. Two models with the same topology differ only in names.

LoRA
: a fine-tuning method that trains small extra matrices instead of all weights.

Zero-shot / few-shot
: prompting with only instructions / with a few solved examples in the prompt.

Constrained decoding
: the generator may only produce tokens that keep the output valid against a JSON schema.

Verify-and-retry
: send the validator's error messages back to the model and ask for a corrected graph.

Self-consistency
: sample several answers and pick the best by the validator's score.

OOD
: out-of-distribution: inputs unlike the training data.

Isomorphic
: two graphs are the same up to renaming the nodes.

Graph edit distance
: the number of node/edge insertions, deletions and substitutions that turn one graph into the other.

B Complete results

All numbers in this appendix are produced by `scripts/aggregate.py` (tables in `results/tables/`) or computed from the raw prediction files (`results/raw/<config>/<set>.jsonl`). Percentages are means over samples; brackets are 95% bootstrap confidence intervals (1,000 resamples). Configuration names are short labels; Table 3 maps them to the configuration ids used in the code and the file names.

B.1 Every inference configuration

Table 3: Every configuration in `configs/experiments.yaml` that produced reported results. "FT" = our merged LoRA model; "syn" = trained with the synthetic data; constr. = JSON-schema constrained decoding; SC = self-consistency (n samples, the validator picks).

Label	id	model	prompt	constr.	decoding	extras	max tok.	eval sets
9B zero-shot	`base9b_zs_c`	Qwen3.5-9B	0-shot	yes	greedy	–	3072	test, novel_topo, ood
9B 3-shot	`base9b_fs_c`	Qwen3.5-9B	3-shot	yes	greedy	–	3072	test, test_anon, novel_topo, ood
2B 3-shot	`base2b_fs_c`	Qwen3.5-2B	3-shot	yes	greedy	–	3072	test, novel_topo, ood
9B 3-shot, unconstrained	`base9b_fs_u`	Qwen3.5-9B	3-shot	no	greedy	–	3072	test
2B FT	`ft2b_c`	qwen35-2b-topograph-merged	0-shot	yes	greedy	–	3072	test, test_anon, novel_topo, ood
9B FT	`ft9b_c`	qwen35-9b-topograph-merged	0-shot	yes	greedy	–	3072	test, test_anon, novel_topo, ood
9B FT, unconstrained	`ft9b_u`	qwen35-9b-topograph-merged	0-shot	no	greedy	–	3072	test, novel_topo, ood
9B FT + retry	`ft9b_c_retry`	qwen35-9b-topograph-merged	0-shot	yes	greedy	retry 2	3072	test, novel_topo, ood
9B 3-shot + retry	`base9b_fs_c_retry`	Qwen3.5-9B	3-shot	yes	greedy	retry 2	3072	test_mini, ood
9B thinking (8k)	`base9b_think`	Qwen3.5-9B	0-shot	no	T=0.6, top-p 0.95	thinking	8192	test_mini, ood
9B thinking (30k)	`base9b_think32k`	Qwen3.5-9B	0-shot	no	T=0.6, top-p 0.95	thinking	30000	test_mini, ood
9B think-then-constrain (30k)	`base9b_think32k_c`	Qwen3.5-9B	0-shot	yes	T=0.6, top-p 0.95	thinking, grammar after think	30000	test_mini, ood
Coder-7B 3-shot	`coder7b_fs_c`	Qwen2.5-Coder-7B-Instruct	3-shot	yes	greedy	–	3072	test, novel_topo, ood
27B 3-shot	`ceil_fs_c`	Qwen3.5-27B-FP8	3-shot	yes	greedy	–	3072	test, novel_topo, ood
27B zero-shot	`ceil_zs_c`	Qwen3.5-27B-FP8	0-shot	yes	greedy	–	3072	test_mini, ood
9B FT, SC-5	`ft9b_sc5`	qwen35-9b-topograph-merged	0-shot	yes	T=0.7	SC 5	3072	test_mini, ood
9B FT + syn	`ft9b_aug_c`	qwen35-9b-topograph-aug-merged	0-shot	yes	greedy	–	3072	test, test_anon, novel_topo, ood, ood2
2B FT + syn	`ft2b_aug_c`	qwen35-2b-topograph-aug-merged	0-shot	yes	greedy	–	3072	test, test_anon, novel_topo, ood, ood2
9B FT, SC-5 + retry	`ft9b_sc5_retry`	qwen35-9b-topograph-merged	0-shot	yes	T=0.7	retry 2, SC 5	3072	test_mini, novel_topo, ood
2B FT + retry	`ft2b_c_retry`	qwen35-2b-topograph-merged	0-shot	yes	greedy	retry 2	3072	test, novel_topo, ood

Label	id	model	prompt	constr.	decoding	extras	max tok.	eval sets
27B zero-shot + retry	`ceil_zs_c_retry`	Qwen3.5-27B-FP8	0-shot	yes	greedy	retry 2	3072	test_mini, novel_topo, ood
27B 3-shot + retry	`ceil_fs_c_retry`	Qwen3.5-27B-FP8	3-shot	yes	greedy	retry 2	3072	test_mini, ood, ood2
9B FT + syn + retry	`ft9b_aug_c_retry`	qwen35-9b-topograph-aug-merged	0-shot	yes	greedy	retry 2	3072	test, novel_topo, ood, ood2
2B FT + syn + retry	`ft2b_aug_c_retry`	qwen35-2b-topograph-aug-merged	0-shot	yes	greedy	retry 2	3072	ood, ood2

B.2 Official test: every headline metric with confidence intervals

Table 4: Official test (1,000 samples), all headline metrics in % (95% CIs for the first three; the CIs of all metrics are in `results/tables/main.csv`).

Config	fully valid	edge F1	exact	isomorphic	graph sim.	PPS	shape acc.
2B 3-shot	1.1 [0.5, 1.8]	62.0 [60.0, 64.1]	6.5 [5.0, 8.0]	7.9	75.2	52.0	84.7
9B zero-shot	19.9 [17.6, 22.5]	61.7 [59.7, 64.0]	20.3 [17.8, 22.8]	30.3	80.1	61.6	79.2
9B 3-shot	54.9 [52.0, 58.1]	94.4 [93.8, 95.0]	57.1 [54.2, 60.2]	57.3	97.2	86.9	99.2
9B 3-shot, unconstrained	54.6 [51.6, 57.7]	94.5 [93.9, 95.0]	57.1 [54.3, 60.2]	57.3	97.2	86.8	99.2
Coder-7B 3-shot	15.4 [13.4, 17.5]	81.5 [80.5, 82.4]	21.7 [19.5, 24.1]	21.7	89.9	70.7	96.9
27B 3-shot	91.0 [89.3, 92.7]	99.8 [99.7, 99.9]	97.9 [97.0, 98.8]	97.9	99.9	99.5	100.0
2B FT	93.3 [91.8, 94.8]	99.4 [99.2, 99.5]	93.7 [92.2, 95.2]	93.7	99.7	97.1	100.0
2B FT + retry	93.2 [91.7, 94.7]	99.4 [99.2, 99.5]	93.6 [92.1, 95.1]	93.6	99.7	97.1	100.0
2B FT + syn	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0	100.0	100.0	100.0
9B FT	99.7 [99.3, 100.0]	100.0 [100.0, 100.0]	99.7 [99.3, 100.0]	99.7	100.0	99.9	100.0
9B FT, unconstrained	99.7 [99.3, 100.0]	100.0 [100.0, 100.0]	99.7 [99.3, 100.0]	99.7	100.0	99.9	100.0
9B FT + retry	99.7 [99.3, 100.0]	100.0 [100.0, 100.0]	99.7 [99.3, 100.0]	99.7	100.0	99.9	100.0
9B FT + syn	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0	100.0	100.0	100.0
9B FT + syn + retry	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	100.0	100.0	100.0	100.0

B.3 Every configuration on every evaluation set

Table 5: Every (configuration, evaluation set) pair that was run, in %. Sets not listed for a configuration were not run.

Config	set	n	fully valid	edge F1	exact	isomorphic	graph sim.
2B 3-shot	test	1000	1.1 [0.5, 1.8]	62.0	6.5 [5.0, 8.0]	7.9	75.2
2B 3-shot	novel_topo	564	1.1 [0.4, 2.0]	57.3	3.2 [1.8, 4.6]	3.7	69.0
2B 3-shot	OOD-1	40	0.0 [0.0, 0.0]	36.7	0.0 [0.0, 0.0]	7.5	62.2
2B 3-shot	OOD-2	21	0.0 [0.0, 0.0]	50.0	4.8 [0.0, 14.3]	14.3	75.3
9B zero-shot	test	1000	19.9 [17.6, 22.5]	61.7	20.3 [17.8, 22.8]	30.3	80.1
9B zero-shot	novel_topo	564	10.3 [8.0, 12.8]	54.3	14.0 [11.3, 16.8]	15.4	71.1
9B zero-shot	OOD-1	40	12.5 [2.5, 22.5]	46.7	5.0 [0.0, 12.5]	15.0	73.8
9B zero-shot	OOD-2	21	38.1 [19.0, 61.9]	64.5	28.6 [14.3, 47.6]	42.9	88.7
9B 3-shot	test	1000	54.9 [52.0, 58.1]	94.4	57.1 [54.2, 60.2]	57.3	97.2
9B 3-shot	test_anon	1000	47.0 [44.1, 50.1]	93.1	50.4 [47.5, 53.5]	50.4	96.5
9B 3-shot	novel_topo	564	32.3 [28.5, 36.0]	91.7	33.9 [29.8, 37.6]	33.9	95.8
9B 3-shot	OOD-1	40	27.5 [15.0, 42.5]	77.5	30.0 [17.4, 45.0]	32.5	84.0
9B 3-shot	OOD-2	21	47.6 [28.6, 66.7]	87.9	38.1 [19.0, 61.9]	47.6	94.8
9B 3-shot, unconstrained	test	1000	54.6 [51.6, 57.7]	94.5	57.1 [54.3, 60.2]	57.3	97.2
9B 3-shot + retry	test_mini	100	67.0 [58.0, 76.0]	94.5	65.0 [55.0, 74.0]	65.0	97.2
9B 3-shot + retry	OOD-1	40	45.0 [29.9, 60.0]	78.4	40.0 [25.0, 55.0]	47.5	84.2
9B 3-shot + retry	OOD-2	21	71.4 [52.4, 90.5]	91.4	66.7 [47.6, 85.7]	76.2	96.9
9B thinking (8k)	test_mini	100	0.0 [0.0, 0.0]	3.0	3.0 [0.0, 7.0]	3.0	3.0
9B thinking (8k)	OOD-1	40	0.0 [0.0, 0.0]	8.7	7.5 [0.0, 17.5]	7.5	8.9
9B thinking (30k)	test_mini	100	7.0 [3.0, 12.0]	65.4	60.0 [50.0, 69.0]	68.0	70.0
9B thinking (30k)	OOD-1	40	10.0 [2.5, 20.0]	71.8	70.0 [55.0, 82.5]	70.0	72.8
9B think-then-constrain (30k)	test_mini	100	42.0 [33.0, 52.0]	44.7	43.0 [34.0, 53.0]	44.0	45.0
9B think-then-constrain (30k)	OOD-1	40	22.5 [10.0, 35.0]	29.7	27.5 [15.0, 42.5]	30.0	31.1
Coder-7B 3-shot	test	1000	15.4 [13.4, 17.5]	81.5	21.7 [19.5, 24.1]	21.7	89.9
Coder-7B 3-shot	novel_topo	564	6.9 [5.0, 8.9]	85.1	19.1 [16.1, 22.3]	19.3	92.0
Coder-7B 3-shot	OOD-1	40	10.0 [2.5, 20.0]	76.4	5.0 [0.0, 12.5]	15.0	89.5

Config	set	n	fully valid	edge F1	exact	isomorphic	graph sim.
Coder-7B 3-shot	OOD-2	21	47.6 [28.6, 66.7]	82.3	33.3 [14.3, 52.4]	38.1	91.4
27B zero-shot	test_mini	100	56.0 [47.0, 66.0]	92.8	65.0 [56.0, 74.0]	65.0	96.4
27B zero-shot	OOD-1	40	67.5 [52.5, 82.5]	89.2	67.5 [52.5, 82.5]	77.5	97.4
27B zero-shot	OOD-2	21	71.4 [52.4, 90.5]	88.9	66.7 [47.6, 85.7]	71.4	94.2
27B zero-shot + retry	test_mini	100	88.0 [82.0, 94.0]	99.1	91.0 [85.0, 96.0]	92.0	99.6
27B zero-shot + retry	novel_topo	564	87.9 [85.3, 90.4]	98.9	90.8 [88.3, 93.1]	90.8	99.4
27B zero-shot + retry	OOD-1	40	100.0 [100.0, 100.0]	97.3	87.5 [77.5, 97.5]	100.0	100.0
27B zero-shot + retry	OOD-2	21	90.5 [76.2, 100.0]	95.2	85.7 [71.4, 100.0]	95.2	98.9
27B 3-shot	test	1000	91.0 [89.3, 92.7]	99.8	97.9 [97.0, 98.8]	97.9	99.9
27B 3-shot	novel_topo	564	92.4 [90.2, 94.3]	99.8	99.1 [98.2, 99.8]	99.1	99.9
27B 3-shot	OOD-1	40	80.0 [67.5, 92.5]	94.2	70.0 [55.0, 82.5]	80.0	95.8
27B 3-shot	OOD-2	21	95.2 [85.7, 100.0]	97.2	90.5 [76.2, 100.0]	95.2	98.9
27B 3-shot + retry	test_mini	100	91.0 [85.0, 96.0]	99.9	99.0 [97.0, 100.0]	99.0	100.0
27B 3-shot + retry	OOD-1	40	95.0 [87.5, 100.0]	99.6	82.5 [70.0, 92.5]	95.0	99.8
27B 3-shot + retry	OOD-2	21	95.2 [85.7, 100.0]	97.2	90.5 [76.2, 100.0]	95.2	98.9
2B FT	test	1000	93.3 [91.8, 94.8]	99.4	93.7 [92.2, 95.2]	93.7	99.7
2B FT	test_anon	1000	93.4 [91.8, 95.0]	99.3	93.9 [92.4, 95.4]	93.9	99.7
2B FT	novel_topo	564	64.4 [60.5, 68.1]	95.7	64.5 [60.6, 68.3]	64.5	97.8
2B FT	OOD-1	40	5.0 [0.0, 12.5]	58.0	2.5 [0.0, 7.5]	5.0	77.2
2B FT	OOD-2	21	28.6 [9.5, 47.6]	77.3	19.0 [4.8, 38.1]	33.3	87.8
2B FT + retry	test	1000	93.2 [91.7, 94.7]	99.4	93.6 [92.1, 95.1]	93.6	99.7
2B FT + retry	novel_topo	564	64.5 [60.8, 68.4]	95.7	64.7 [61.0, 68.6]	64.7	97.8
2B FT + retry	OOD-1	40	7.5 [0.0, 17.5]	57.9	2.5 [0.0, 7.5]	5.0	77.4
2B FT + syn	test	1000	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
2B FT + syn	test_anon	1000	99.8 [99.5, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
2B FT + syn	novel_topo	564	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
2B FT + syn	OOD-1	40	72.5 [57.5, 87.5]	94.0	70.0 [55.0, 85.0]	72.5	96.7
2B FT + syn	OOD-2	21	71.4 [52.4, 90.5]	89.7	52.4 [33.3, 71.4]	57.1	94.8
2B FT + syn + retry	OOD-1	40	72.5 [57.5, 87.5]	94.0	70.0 [55.0, 85.0]	72.5	96.7
2B FT + syn + retry	OOD-2	21	66.7 [47.6, 85.7]	89.7	52.4 [33.3, 71.4]	57.1	94.8
9B FT	test	1000	99.7 [99.3, 100.0]	100.0	99.7 [99.3, 100.0]	99.7	100.0
9B FT	test_anon	1000	99.6 [99.1, 99.9]	100.0	99.8 [99.5, 100.0]	99.8	100.0
9B FT	novel_topo	564	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT	OOD-1	40	47.5 [32.5, 62.5]	82.0	30.0 [17.5, 42.5]	57.5	94.0
9B FT	OOD-2	21	85.7 [71.4, 100.0]	94.5	71.4 [52.4, 90.5]	81.0	97.7
9B FT, unconstrained	test	1000	99.7 [99.3, 100.0]	100.0	99.7 [99.3, 100.0]	99.7	100.0
9B FT, unconstrained	novel_topo	564	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT, unconstrained	OOD-1	40	45.0 [30.0, 60.0]	81.8	30.0 [17.5, 42.5]	55.0	93.9
9B FT + retry	test	1000	99.7 [99.3, 100.0]	100.0	99.7 [99.3, 100.0]	99.7	100.0
9B FT + retry	novel_topo	564	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT + retry	OOD-1	40	67.5 [52.5, 82.5]	85.9	45.0 [30.0, 60.0]	70.0	94.3
9B FT + retry	OOD-2	21	90.5 [76.2, 100.0]	95.1	76.2 [57.1, 95.2]	85.7	98.0
9B FT, SC-5	test_mini	100	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT, SC-5	OOD-1	40	50.0 [35.0, 65.0]	82.5	32.5 [20.0, 47.5]	57.5	94.6
9B FT, SC-5 + retry	test_mini	100	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT, SC-5 + retry	novel_topo	564	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT, SC-5 + retry	OOD-1	40	67.5 [52.5, 82.5]	87.6	42.5 [27.5, 57.5]	70.0	96.7
9B FT + syn	test	1000	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT + syn	test_anon	1000	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT + syn	novel_topo	564	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT + syn	OOD-1	40	95.0 [87.5, 100.0]	99.9	97.5 [92.5, 100.0]	97.5	99.9
9B FT + syn	OOD-2	21	76.2 [57.1, 90.5]	95.5	76.2 [57.1, 90.6]	85.7	98.1
9B FT + syn + retry	test	1000	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT + syn + retry	novel_topo	564	100.0 [100.0, 100.0]	100.0	100.0 [100.0, 100.0]	100.0	100.0
9B FT + syn + retry	OOD-1	40	97.5 [92.5, 100.0]	99.9	97.5 [92.5, 100.0]	97.5	99.9
9B FT + syn + retry	OOD-2	21	90.5 [76.2, 100.0]	95.5	76.2 [57.1, 90.6]	85.7	98.1

B.4 By complexity tier (official test)

Table 6: Official test by tier (200 samples each), in %. L0 = neuron MLP, L1 = chain, L2 = residual, L3 = branch-concat, L4 = nested.

	fully valid					exact					edge F1				
Config	L0	L1	L2	L3	L4	L0	L1	L2	L3	L4	L0	L1	L2	L3	L4
2B 3-shot	0.0	5.0	0.0	0.0	0.5	0.0	32.0	0.0	0.0	0.5	65.9	67.9	56.3	37.2	82.5
9B zero-shot	44.5	48.5	0.0	6.5	0.0	18.0	78.0	0.0	5.5	0.0	64.0	94.4	48.9	65.4	36.0
9B 3-shot	93.0	77.5	28.0	59.0	17.0	93.5	90.0	24.5	60.5	17.0	98.6	99.3	89.4	91.9	92.6
9B 3-shot, unconstrained	93.5	77.0	25.5	60.5	16.5	94.0	90.0	23.5	61.5	16.5	99.0	99.3	89.3	92.2	92.4
Coder-7B 3-shot	16.0	19.5	13.5	5.0	23.0	23.0	45.5	13.5	5.0	21.5	76.1	94.0	75.0	78.8	83.5
27B 3-shot	92.0	72.5	95.5	97.5	97.5	100.0	99.0	93.0	100.0	97.5	100.0	100.0	99.5	100.0	99.4
2B FT	100.0	94.5	100.0	98.0	74.0	100.0	96.5	100.0	98.0	74.0	100.0	99.9	100.0	99.6	97.3
2B FT + retry	100.0	94.0	100.0	98.0	74.0	100.0	96.0	100.0	98.0	74.0	100.0	99.9	100.0	99.6	97.3
2B FT + syn	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

Config	fully valid					exact					edge F1				
	L0	L1	L2	L3	L4	L0	L1	L2	L3	L4	L0	L1	L2	L3	L4
9B FT	100.0	98.5	100.0	100.0	100.0	100.0	98.5	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT, unconstrained	100.0	98.5	100.0	100.0	100.0	100.0	98.5	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT + retry	100.0	98.5	100.0	100.0	100.0	100.0	98.5	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT + syn	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT + syn + retry	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

B.5 By graph size (official test)

Table 7: Official test by number of reference nodes, in %.

Config (nodes)	fully valid				exact			
	≤10	11–15	16–20	>20	≤10	11–15	16–20	>20
2B 3-shot	1.4	0.0	2.2	0.0	6.8	9.1	5.1	2.1
9B zero-shot	34.2	27.7	4.7	2.1	26.4	17.2	17.4	20.8
9B 3-shot	84.2	63.5	31.3	16.7	83.6	66.2	34.5	22.9
9B 3-shot, unconstrained	83.6	63.5	30.7	17.7	82.9	66.2	35.1	22.9
Coder-7B 3-shot	27.7	5.4	18.0	0.0	31.8	12.2	22.8	16.7
27B 3-shot	99.7	91.9	87.7	72.9	100.0	100.0	98.4	83.3
2B FT	100.0	100.0	80.4	94.8	100.0	100.0	81.6	94.8
2B FT + retry	100.0	100.0	80.4	93.8	100.0	100.0	81.6	93.8
2B FT + syn	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT	100.0	100.0	100.0	96.9	100.0	100.0	100.0	96.9
9B FT, unconstrained	100.0	100.0	100.0	96.9	100.0	100.0	100.0	96.9
9B FT + retry	100.0	100.0	100.0	96.9	100.0	100.0	100.0	96.9
9B FT + syn	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT + syn + retry	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

B.6 By augmentation variant (robustness to renaming and dead code)

Table 8: Official test by variant (250 samples each), in %: the original, the forward argument renamed, dead code inserted, and all three changes plus a shuffled `__init__`.

Config	fully valid				exact			
	orig.	renamed	dead code	all three	orig.	renamed	dead code	all three
2B 3-shot	2.0	2.0	0.4	0.0	8.4	6.4	6.8	4.4
9B zero-shot	22.8	17.6	21.6	17.6	19.2	17.6	23.2	21.2
9B 3-shot	63.2	57.6	52.4	46.4	65.2	57.2	56.0	50.0
9B 3-shot, unconstrained	63.2	57.2	51.2	46.8	64.8	58.0	55.6	50.0
Coder-7B 3-shot	24.4	23.6	7.6	6.0	34.0	33.2	10.8	8.8
27B 3-shot	90.4	90.4	91.6	91.6	98.0	97.2	99.2	97.2
2B FT	93.6	93.2	94.0	92.4	94.0	93.2	94.4	93.2
2B FT + retry	93.6	93.2	93.6	92.4	94.0	93.2	94.0	93.2
2B FT + syn	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT	100.0	100.0	99.6	99.2	100.0	100.0	99.6	99.2
9B FT, unconstrained	100.0	100.0	99.6	99.2	100.0	100.0	99.6	99.2
9B FT + retry	100.0	99.6	99.6	99.6	100.0	99.6	99.6	99.6
9B FT + syn	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0
9B FT + syn + retry	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0

B.7 Seen versus held-out topologies

Table 9: From familiar to unfamiliar, in %: official-test samples whose topology occurs in train_ft (800), official-test samples with a held-out topology (200), novel_topo (564), OOD-1 (40), OOD-2 (21). "–" = not run. NaN-free cells only.

Config	fully valid					exact				
	seen	held-out	novel	OOD-1	OOD-2	seen	held-out	novel	OOD-1	OOD-2
2B 3-shot	1.2	0.5	1.1	0.0	0.0	7.6	2.0	3.2	0.0	4.8
9B zero-shot	22.8	8.5	10.3	12.5	38.1	22.5	11.5	14.0	5.0	28.6
9B 3-shot	60.4	33.0	32.3	27.5	47.6	62.9	34.0	33.9	30.0	38.1
9B 3-shot, unconstrained	60.4	31.5	–	–	–	62.9	34.0	–	–	–
9B 3-shot + retry	–	–	–	45.0	71.4	–	–	–	40.0	66.7
9B thinking (8k)	–	–	–	0.0	–	–	–	–	7.5	–
9B thinking (30k)	–	–	–	10.0	–	–	–	–	70.0	–
9B think-then-constrain (30k)	–	–	–	22.5	–	–	–	–	27.5	–

Config	fully valid					exact				
	seen	held-out	novel	OOD-1	OOD-2	seen	held-out	novel	OOD-1	OOD-2
Coder-7B 3-shot	16.1	12.5	6.9	10.0	47.6	22.2	19.5	19.1	5.0	33.3
27B zero-shot	–	–	–	67.5	71.4	–	–	–	67.5	66.7
27B zero-shot + retry	–	–	87.9	100.0	90.5	–	–	90.8	87.5	85.7
27B 3-shot	90.4	93.5	92.4	80.0	95.2	98.1	97.0	99.1	70.0	90.5
27B 3-shot + retry	–	–	–	95.0	95.2	–	–	–	82.5	90.5
2B FT	99.2	69.5	64.4	5.0	28.6	99.6	70.0	64.5	2.5	19.0
2B FT + retry	99.1	69.5	64.5	7.5	–	99.5	70.0	64.7	2.5	–
2B FT + syn	100.0	100.0	100.0	72.5	71.4	100.0	100.0	100.0	70.0	52.4
2B FT + syn + retry	–	–	–	72.5	66.7	–	–	–	70.0	52.4
9B FT	99.9	99.0	100.0	47.5	85.7	99.9	99.0	100.0	30.0	71.4
9B FT, unconstrained	99.9	99.0	100.0	45.0	–	99.9	99.0	100.0	30.0	–
9B FT + retry	100.0	98.5	100.0	67.5	90.5	100.0	98.5	100.0	45.0	76.2
9B FT, SC-5	–	–	–	50.0	–	–	–	–	32.5	–
9B FT, SC-5 + retry	–	–	100.0	67.5	–	–	–	100.0	42.5	–
9B FT + syn	100.0	100.0	100.0	95.0	76.2	100.0	100.0	100.0	97.5	76.2
9B FT + syn + retry	100.0	100.0	100.0	97.5	90.5	100.0	100.0	100.0	97.5	76.2

B.8 OOD-1 and OOD-2 by category

Category	9B zs	9B 3s	C7B 3s	27B 3s	27B zs+r	2B FT	2B syn	9B FT	9B FT+r	9B syn	9B syn+r
bottleneck	25/25	25/25	0/0	100/100	100/100	0/0	75/75	75/75	100/100	100/100	100/100
densenet	25/0	25/25	0/0	100/100	100/75	0/0	75/75	25/0	50/25	100/100	100/100
inception	0/0	25/25	0/0	75/75	100/100	0/0	25/25	50/50	75/75	100/100	100/100
mlp_norm	0/0	75/100	25/25	75/75	100/100	25/25	75/25	75/75	75/100	100/100	100/100
modulelist	25/0	0/0	0/0	75/75	100/75	0/0	100/100	0/0	25/25	100/100	100/100
nested_basicblock	0/0	0/0	0/0	25/25	100/75	0/0	75/50	0/0	50/25	100/100	100/100
squeeze_excite	0/0	0/25	0/0	75/75	100/100	0/0	75/50	25/0	50/0	100/100	100/100
two_input	0/0	25/25	0/0	100/75	100/100	0/0	100/100	75/0	75/0	100/100	100/100
unet	0/25	50/50	0/0	75/75	100/100	0/0	100/100	75/75	75/75	75/75	75/75
vgg	50/0	50/25	75/25	100/25	100/50	25/0	25/100	75/25	100/25	75/100	100/100

Table 10: OOD-1 (4 modules per category): fully valid / exact match, in %. Column key: zs = zero-shot, 3s = 3-shot, +r = with retry, C7B = Coder-7B, syn = fine-tuned with synthetic data.

Category	9B zs	9B 3s	C7B 3s	27B 3s	27B zs+r	2B FT	2B syn	9B FT	9B FT+r	9B syn	9B syn+r
functional_pool_reshape	67/67	100/67	100/100	100/100	100/100	33/67	100/100	100/100	100/100	100/100	100/100
groupnorm_prelu	33/33	33/33	0/0	100/100	100/100	0/0	67/67	100/100	100/100	100/67	100/67
matmul_attention	0/0	0/0	33/33	67/100	67/67	0/0	33/0	100/67	100/67	67/67	100/67
mixed_functional	0/0	67/67	0/0	100/100	100/100	0/0	67/67	67/100	67/100	67/67	67/67
scalar_arith	67/0	33/33	67/0	100/67	67/67	100/0	100/0	67/0	100/33	67/67	100/67
stack_reduce	67/33	67/33	100/67	100/67	100/67	33/33	100/67	100/33	100/33	100/67	100/67
transposed_conv	33/67	33/33	33/33	100/100	100/100	33/33	33/67	67/100	67/100	33/100	67/100

Table 11: OOD-2 (3 modules per category): fully valid / exact match, in %. Column key: zs = zero-shot, 3s = 3-shot, +r = with retry, C7B = Coder-7B, syn = fine-tuned with synthetic data.

B.9 Validity levels and code-consistency rates

Table 12: Share of outputs passing each validator level (each level counted on its own), in %. "repaired" = the parser had to fix the JSON (e.g. close a truncated bracket); "halluc." = share of predicted `call_module` nodes that are not declared layers; "dead incl." = share of dead-code samples whose dead layer appears in the prediction.

Config	set	parse	schema	repaired	V1	V2	V3	V4	valid	halluc.	dead incl.
2B 3-shot	test	100.0	90.0	19.8	21.5	19.1	79.2	70.2	1.1	0.9	6.0
2B 3-shot	novel_topo	100.0	83.7	29.6	12.6	15.4	74.6	61.5	1.1	1.2	1.8
2B 3-shot	OOD-1	100.0	90.0	15.0	10.0	22.5	50.0	32.5	0.0	17.8	–
2B 3-shot	OOD-2	100.0	100.0	0.0	0.0	38.1	85.7	66.7	0.0	0.0	–
9B zero-shot	test	100.0	87.3	22.0	25.0	34.0	82.8	73.2	19.9	0.3	1.0
9B zero-shot	novel_topo	100.0	78.5	35.5	11.2	15.6	76.6	67.6	10.3	0.3	0.4
9B zero-shot	OOD-1	100.0	95.0	10.0	30.0	37.5	77.5	65.0	12.5	12.1	–
9B zero-shot	OOD-2	100.0	100.0	4.8	57.1	61.9	95.2	81.0	38.1	3.6	–
9B 3-shot	test	100.0	100.0	0.0	64.9	65.2	99.7	84.4	54.9	0.1	0.0
9B 3-shot	test_anon	100.0	100.0	0.0	59.8	59.9	99.5	82.9	47.0	0.0	0.0
9B 3-shot	novel_topo	100.0	100.0	0.0	38.7	37.1	100.0	74.1	32.3	0.0	0.0
9B 3-shot	OOD-1	100.0	100.0	0.0	55.0	52.5	80.0	75.0	27.5	9.4	–
9B 3-shot	OOD-2	100.0	100.0	0.0	61.9	66.7	95.2	95.2	47.6	0.0	–
9B 3-shot, unconstrained	test	100.0	100.0	0.0	64.9	65.0	99.7	84.9	54.6	0.1	0.0
9B 3-shot + retry	test_mini	100.0	100.0	0.0	73.0	74.0	100.0	85.0	67.0	0.0	0.0
9B 3-shot + retry	OOD-1	100.0	100.0	0.0	82.5	62.5	80.0	80.0	45.0	7.6	–
9B 3-shot + retry	OOD-2	100.0	100.0	0.0	85.7	85.7	100.0	95.2	71.4	0.0	–
9B thinking (8k)	test_mini	4.0	0.0	1.0	3.0	3.0	3.0	3.0	0.0	0.0	0.0
9B thinking (8k)	OOD-1	10.0	0.0	0.0	10.0	10.0	7.5	10.0	0.0	6.2	–
9B thinking (30k)	test_mini	72.0	7.0	1.0	70.0	70.0	71.0	71.0	7.0	0.0	0.0
9B thinking (30k)	OOD-1	77.5	10.0	0.0	77.5	77.5	70.0	77.5	10.0	3.0	–
9B think-then-constrain (30k)	test_mini	75.0	45.0	30.0	44.0	45.0	45.0	43.0	42.0	0.0	0.0
9B think-then-constrain (30k)	OOD-1	67.5	35.0	32.5	32.5	35.0	27.5	27.5	22.5	6.5	–

Config	set	parse	schema	repaired	V1	V2	V3	V4	valid	halluc.	dead incl.
Coder-7B 3-shot	test	100.0	99.6	0.7	30.2	34.1	79.1	64.2	15.4	0.2	34.6
Coder-7B 3-shot	novel_topo	100.0	100.0	0.0	22.7	28.7	77.8	51.4	6.9	0.7	30.1
Coder-7B 3-shot	OOD-1	100.0	100.0	0.0	27.5	20.0	90.0	75.0	10.0	7.4	–
Coder-7B 3-shot	OOD-2	100.0	100.0	0.0	81.0	61.9	95.2	95.2	47.6	0.0	–
27B zero-shot	test_mini	100.0	100.0	0.0	60.0	67.0	100.0	73.0	56.0	0.0	0.0
27B zero-shot	OOD-1	100.0	100.0	0.0	80.0	82.5	92.5	97.5	67.5	5.8	–
27B zero-shot	OOD-2	100.0	100.0	0.0	71.4	85.7	100.0	90.5	71.4	0.0	–
27B zero-shot + retry	test_mini	100.0	100.0	0.0	88.0	97.0	100.0	99.0	88.0	0.0	0.0
27B zero-shot + retry	novel_topo	100.0	100.0	0.0	88.7	94.9	99.6	97.9	87.9	0.0	0.0
27B zero-shot + retry	OOD-1	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	–
27B zero-shot + retry	OOD-2	100.0	100.0	0.0	90.5	100.0	100.0	100.0	90.5	0.0	–
27B 3-shot	test	100.0	100.0	0.0	91.8	98.5	100.0	99.1	91.0	0.0	0.0
27B 3-shot	novel_topo	100.0	100.0	0.0	93.8	99.1	100.0	97.9	92.4	0.0	0.0
27B 3-shot	OOD-1	100.0	100.0	0.0	90.0	87.5	92.5	97.5	80.0	1.4	–
27B 3-shot	OOD-2	100.0	100.0	0.0	95.2	100.0	100.0	100.0	95.2	0.0	–
27B 3-shot + retry	test_mini	100.0	100.0	0.0	91.0	100.0	100.0	100.0	91.0	0.0	0.0
27B 3-shot + retry	OOD-1	100.0	100.0	0.0	97.5	97.5	100.0	97.5	95.0	0.0	–
27B 3-shot + retry	OOD-2	100.0	100.0	0.0	95.2	100.0	100.0	100.0	95.2	0.0	–
2B FT	test	100.0	100.0	0.0	94.5	93.7	100.0	100.0	93.3	0.0	0.0
2B FT	test_anon	100.0	100.0	0.0	94.6	94.0	99.9	99.9	93.4	0.0	0.0
2B FT	novel_topo	100.0	100.0	0.0	70.6	64.5	100.0	100.0	64.4	0.0	0.0
2B FT	OOD-1	100.0	100.0	0.0	55.0	35.0	70.0	57.5	5.0	9.2	–
2B FT	OOD-2	100.0	100.0	0.0	71.4	52.4	90.5	90.5	28.6	5.1	–
2B FT + retry	test	100.0	100.0	0.0	94.4	93.6	100.0	100.0	93.2	0.0	0.0
2B FT + retry	novel_topo	100.0	100.0	0.0	69.7	64.7	100.0	100.0	64.5	0.0	0.0
2B FT + retry	OOD-1	100.0	100.0	0.0	60.0	40.0	72.5	62.5	7.5	9.6	–
2B FT + syn	test	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
2B FT + syn	test_anon	100.0	100.0	0.0	99.8	100.0	100.0	100.0	99.8	0.0	0.0
2B FT + syn	novel_topo	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
2B FT + syn	OOD-1	100.0	100.0	0.0	92.5	92.5	95.0	82.5	72.5	0.0	–
2B FT + syn	OOD-2	100.0	100.0	0.0	95.2	85.7	100.0	81.0	71.4	0.0	–
2B FT + syn + retry	OOD-1	100.0	100.0	0.0	92.5	92.5	95.0	82.5	72.5	0.0	–
2B FT + syn + retry	OOD-2	100.0	100.0	0.0	90.5	85.7	100.0	81.0	66.7	0.0	–
9B FT	test	100.0	100.0	0.0	99.7	99.7	100.0	100.0	99.7	0.0	0.0
9B FT	test_anon	100.0	100.0	0.0	99.8	99.8	100.0	99.8	99.6	0.0	0.0
9B FT	novel_topo	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT	OOD-1	100.0	100.0	0.0	90.0	80.0	80.0	70.0	47.5	11.0	–
9B FT	OOD-2	100.0	100.0	0.0	95.2	95.2	100.0	90.5	85.7	0.0	–
9B FT, unconstrained	test	100.0	100.0	0.0	99.7	99.7	100.0	100.0	99.7	0.0	0.0
9B FT, unconstrained	novel_topo	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT, unconstrained	OOD-1	100.0	100.0	0.0	87.5	77.5	80.0	70.0	45.0	11.0	–
9B FT + retry	test	100.0	100.0	0.0	99.7	99.7	100.0	100.0	99.7	0.0	0.0
9B FT + retry	novel_topo	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT + retry	OOD-1	100.0	100.0	0.0	97.5	97.5	87.5	82.5	67.5	5.9	–
9B FT + retry	OOD-2	100.0	100.0	0.0	100.0	100.0	100.0	90.5	90.5	0.0	–
9B FT, SC-5	test_mini	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT, SC-5	OOD-1	100.0	100.0	0.0	92.5	82.5	80.0	75.0	50.0	13.2	–
9B FT, SC-5 + retry	test_mini	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT, SC-5 + retry	novel_topo	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT, SC-5 + retry	OOD-1	100.0	100.0	0.0	95.0	95.0	87.5	87.5	67.5	9.7	–
9B FT + syn	test	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT + syn	test_anon	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT + syn	novel_topo	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT + syn	OOD-1	100.0	100.0	0.0	100.0	100.0	100.0	95.0	95.0	0.0	–
9B FT + syn	OOD-2	100.0	100.0	0.0	100.0	100.0	100.0	76.2	76.2	0.0	–
9B FT + syn + retry	test	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT + syn + retry	novel_topo	100.0	100.0	0.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0
9B FT + syn + retry	OOD-1	100.0	100.0	0.0	100.0	100.0	100.0	97.5	97.5	0.0	–
9B FT + syn + retry	OOD-2	100.0	100.0	0.0	100.0	100.0	100.0	90.5	90.5	0.0	–

B.10 Verify-and-retry, every run

Table 13: Verify-and-retry (at most 2 extra rounds; only invalid graphs are retried). "Before" is the first attempt inside the retry run. It can differ slightly from the separate run without retry (e.g. 2B FT + syn on OOD-2: 66.7 vs 71.4), because vLLM's greedy decoding is not bit-identical when the batch composition changes (see Section B.15).

Config	set	n	valid before	valid after	gain	mean rounds
9B 3-shot + retry	test_mini	100	50.0	67.0	+17.0	0.91
9B 3-shot + retry	OOD-1	40	27.5	45.0	+17.5	1.27
9B 3-shot + retry	OOD-2	21	47.6	71.4	+23.8	0.86
27B zero-shot + retry	test_mini	100	55.0	88.0	+33.0	0.65
27B zero-shot + retry	novel_topo	564	44.1	87.9	+43.8	0.79
27B zero-shot + retry	OOD-1	40	70.0	100.0	+30.0	0.33
27B zero-shot + retry	OOD-2	21	71.4	90.5	+19.0	0.43
27B 3-shot + retry	test_mini	100	86.0	91.0	+5.0	0.23
27B 3-shot + retry	OOD-1	40	80.0	95.0	+15.0	0.28
27B 3-shot + retry	OOD-2	21	95.2	95.2	+0.0	0.10
2B FT + retry	test	1000	93.2	93.2	+0.0	0.14
2B FT + retry	novel_topo	564	64.2	64.5	+0.4	0.71
2B FT + retry	OOD-1	40	5.0	7.5	+2.5	1.90
2B FT + syn + retry	OOD-1	40	72.5	72.5	+0.0	0.55
2B FT + syn + retry	OOD-2	21	66.7	66.7	+0.0	0.67
9B FT + retry	test	1000	99.6	99.7	+0.1	0.01
9B FT + retry	novel_topo	564	100.0	100.0	+0.0	0.00

Config	set	n	valid before	valid after	gain	mean rounds
9B FT + retry	OOD-1	40	47.5	67.5	+20.0	0.93
9B FT + retry	OOD-2	21	85.7	90.5	+4.8	0.24
9B FT, SC-5 + retry	test_mini	100	100.0	100.0	+0.0	0.00
9B FT, SC-5 + retry	novel_topo	564	100.0	100.0	+0.0	0.00
9B FT, SC-5 + retry	OOD-1	40	50.0	67.5	+17.5	0.85
9B FT + syn + retry	test	1000	100.0	100.0	+0.0	0.00
9B FT + syn + retry	novel_topo	564	100.0	100.0	+0.0	0.00
9B FT + syn + retry	OOD-1	40	95.0	97.5	+2.5	0.07
9B FT + syn + retry	OOD-2	21	76.2	90.5	+14.3	0.38

B.11 Topological properties preserved (official test)

Property	9B zs	9B 3s	C7B 3s	27B 3s	2B FT	2B syn	9B FT	9B FT+r	9B syn	9B syn+r
n_nodes	76	100	78	100	100	100	100	100	100	100
n_edges	36	72	36	99	94	100	100	100	100	100
n_sources	76	96	85	99	95	100	100	100	100	100
n_sinks	54	77	71	99	95	100	100	100	100	100
n_branch	55	87	69	100	100	100	100	100	100	100
n_merge	41	78	60	100	95	100	100	100	100	100
depth	44	74	38	98	96	100	100	100	100	100
width	59	88	63	99	98	100	100	100	100	100
n_skip_edges	48	77	62	100	95	100	100	100	100	100
n_paths_in_out	35	72	43	99	94	100	100	100	100	100
n_cycles	87	100	100	100	100	100	100	100	100	100
hierarchy_groups	87	100	99	100	100	100	100	100	100	100
is_dag	87	100	100	100	100	100	100	100	100	100
weakly_connected	76	96	85	99	95	100	100	100	100	100
PPS (mean)	62	87	71	100	97	100	100	100	100	100

Table 14: Share of test samples whose predicted graph has exactly the reference's value of each property, in %. Definitions in Table 26.

B.12 Failure labels (official test)

Label	9B zs	9B 3s	C7B 3s	27B 3s	2B FT	9B FT	9B FT+r
parse_fail	127	0	4	0	0	0	0
missing_skip_edge	391	240	377	11	56	0	0
missing_edge	529	362	766	20	46	3	3
extra_edge	498	356	731	19	56	0	0
missing_node	115	7	13	0	0	0	0
hallucinated_node	130	2	210	0	0	0	0
dead_code_included	5	0	173	0	0	0	0
wrong_kind	2	0	26	0	0	0	0
naming_only	100	2	0	0	0	0	0
shape_only	35	16	28	3	0	0	0
order_error	1	2	0	0	0	0	0
cycle	0	0	0	0	0	0	0

Table 15: Number of official-test samples (out of 1,000) with each failure label; a sample can have several. Definitions in Table 27.

B.13 Checking the edit-distance metric against the true graph edit distance

Table 16: True graph edit distance (networkx, 2 s budget per pair) vs our aligned edit distance (AED) on a stratified 100-sample test subset per configuration (pairs that are identical are included). "GED exact" = the search finished within the budget (otherwise the value is an upper bound). AED is never below an exactly computed GED, as it should be (it is the cost of one specific edit path). "–" = the correlation is undefined (all distances equal).

Config	pairs	GED exact	Spearman	AED≥GED	mean AED	mean GED
2B 3-shot	89	40.4	0.863	100.0	10.18	8.44
9B zero-shot	89	66.3	0.969	100.0	4.93	3.60
9B 3-shot	100	75.0	0.993	100.0	2.20	2.08
9B 3-shot, unconstrained	100	76.0	0.993	100.0	2.02	1.90
Coder-7B 3-shot	100	50.0	0.986	100.0	5.93	5.53
27B 3-shot	100	99.0	1.000	100.0	0.09	0.09
2B FT	100	97.0	1.000	100.0	0.11	0.11
2B FT + retry	100	97.0	1.000	100.0	0.11	0.11

Config	pairs	GED exact	Spearman	AED≥GED	mean AED	mean GED
2B FT + syn	100	100.0	–	100.0	0.00	0.00
9B FT	100	100.0	1.000	100.0	0.01	0.01
9B FT, unconstrained	100	100.0	1.000	100.0	0.01	0.01
9B FT + retry	100	100.0	1.000	100.0	0.01	0.01
9B FT + syn	100	100.0	–	100.0	0.00	0.00
9B FT + syn + retry	100	100.0	–	100.0	0.00	0.00
all	1378	86.3	0.998	100.0	1.74	1.49

B.14 Generation statistics: prompt and answer lengths, truncation, speed

Table 17: From the raw prediction files. Answer tokens include every retry round and every self-consistency sample. "truncated" = the answer hit the token limit (`finish_reason = length`). Seconds per sample are amortised over the whole batch on one L40S (vLLM processes many prompts in parallel), so they measure throughput, not the latency of a single request (a single 9B request takes about 9 s in the app).

Config	set	n	prompt tok.	answer tok. (mean)	max	truncated	s/sample	rounds
2B 3-shot	test	1000	2726	938	3072	19.8	0.28	0.00
2B 3-shot	novel_topo	564	2745	1219	3072	29.6	0.31	0.00
2B 3-shot	OOD-1	40	2599	790	3072	15.0	0.72	0.00
2B 3-shot	OOD-2	21	2522	237	415	0.0	0.44	0.00
9B zero-shot	test	1000	874	1016	3072	22.0	0.59	0.00
9B zero-shot	novel_topo	564	893	1388	3072	35.5	0.83	0.00
9B zero-shot	OOD-1	40	747	665	3072	10.0	2.11	0.00
9B zero-shot	OOD-2	21	670	377	3072	4.8	3.78	0.00
9B 3-shot	test	1000	2726	480	963	0.0	0.52	0.00
9B 3-shot	test_anon	1000	2717	480	964	0.0	0.51	0.00
9B 3-shot	novel_topo	564	2745	519	871	0.0	0.56	0.00
9B 3-shot	OOD-1	40	2599	353	643	0.0	0.78	0.00
9B 3-shot	OOD-2	21	2522	224	345	0.0	0.91	0.00
9B 3-shot	untraceable	6	2510	159	289	0.0	2.35	0.00
9B 3-shot, unconstrained	test	1000	2726	481	961	0.0	0.51	0.00
9B 3-shot + retry	test_mini	100	2732	1021	2823	0.0	0.67	0.91
9B 3-shot + retry	OOD-1	40	2599	817	1953	0.0	0.72	1.27
9B 3-shot + retry	OOD-2	21	2522	445	1009	0.0	0.83	0.86
9B thinking (8k)	test_mini	100	878	8165	8192	97.0	6.92	0.00
9B thinking (8k)	OOD-1	40	745	8162	8192	90.0	8.52	0.00
9B thinking (30k)	test_mini	100	878	17214	30000	28.0	27.40	0.00
9B thinking (30k)	OOD-1	40	745	15786	30000	22.5	29.47	0.00
9B think-then-constrain (30k)	test_mini	100	878	22352	30000	55.0	35.19	0.00
9B think-then-constrain (30k)	OOD-1	40	745	23586	30000	65.0	43.71	0.00
Coder-7B 3-shot	test	1000	2638	508	3072	0.7	0.58	0.00
Coder-7B 3-shot	novel_topo	564	2657	530	950	0.0	0.56	0.00
Coder-7B 3-shot	OOD-1	40	2518	415	1070	0.0	0.87	0.00
Coder-7B 3-shot	OOD-2	21	2446	222	351	0.0	0.64	0.00
27B zero-shot	test_mini	100	880	490	958	0.0	1.22	0.00
27B zero-shot	OOD-1	40	747	393	724	0.0	1.25	0.00
27B zero-shot	OOD-2	21	670	225	334	0.0	1.33	0.00
27B zero-shot + retry	test_mini	100	880	895	2790	0.0	1.30	0.65
27B zero-shot + retry	novel_topo	564	893	996	2090	0.0	1.08	0.79
27B zero-shot + retry	OOD-1	40	747	554	2185	0.0	1.27	0.33
27B zero-shot + retry	OOD-2	21	670	344	962	0.0	1.33	0.43
27B 3-shot	test	1000	2726	479	940	0.0	1.77	0.00
27B 3-shot	novel_topo	564	2745	519	871	0.0	1.86	0.00
27B 3-shot	OOD-1	40	2599	380	725	0.0	1.83	0.00
27B 3-shot	OOD-2	21	2522	230	355	0.0	1.80	0.00
27B 3-shot + retry	test_mini	100	2732	635	2685	0.0	2.01	0.23
27B 3-shot + retry	OOD-1	40	2599	531	2175	0.0	1.84	0.28
27B 3-shot + retry	OOD-2	21	2522	264	1065	0.0	1.55	0.10
2B FT	test	1000	874	478	940	0.0	0.08	0.00
2B FT	test_anon	1000	865	478	940	0.0	0.08	0.00
2B FT	novel_topo	564	893	515	859	0.0	0.08	0.00
2B FT	OOD-1	40	747	334	638	0.0	0.14	0.00
2B FT	OOD-2	21	670	224	327	0.0	0.26	0.00
2B FT	untraceable	6	658	197	316	0.0	0.64	0.00
2B FT + retry	test	1000	874	567	2649	0.0	0.08	0.14
2B FT + retry	novel_topo	564	893	975	2577	0.0	0.08	0.71
2B FT + retry	OOD-1	40	747	996	1914	0.0	0.14	1.90
2B FT + syn	test	1000	874	479	940	0.0	0.08	0.00
2B FT + syn	test_anon	1000	865	479	940	0.0	0.08	0.00

Config	set	n	prompt tok.	answer tok. (mean)	max	truncated	s/sample	rounds
2B FT + syn	novel_topo	564	893	518	865	0.0	0.08	0.00
2B FT + syn	OOD-1	40	747	389	688	0.0	0.16	0.00
2B FT + syn	OOD-2	21	670	224	345	0.0	0.32	0.00
2B FT + syn + retry	OOD-1	40	747	607	2064	0.0	0.21	0.55
2B FT + syn + retry	OOD-2	21	670	382	969	0.0	0.14	0.67
9B FT	test	1000	874	479	940	0.0	0.29	0.00
9B FT	test_anon	1000	865	479	940	0.0	0.28	0.00
9B FT	novel_topo	564	893	518	865	0.0	0.29	0.00
9B FT	OOD-1	40	747	368	624	0.0	0.53	0.00
9B FT	OOD-2	21	670	230	349	0.0	0.71	0.00
9B FT	untraceable	6	658	180	287	0.0	2.28	0.00
9B FT, unconstrained	test	1000	874	479	940	0.0	0.27	0.00
9B FT, unconstrained	novel_topo	564	893	518	865	0.0	0.28	0.00
9B FT, unconstrained	OOD-1	40	747	368	624	0.0	0.53	0.00
9B FT + retry	test	1000	874	485	2577	0.0	0.28	0.01
9B FT + retry	novel_topo	564	893	518	865	0.0	0.30	0.00
9B FT + retry	OOD-1	40	747	739	1882	0.0	0.53	0.93
9B FT + retry	OOD-2	21	670	287	972	0.0	0.77	0.24
9B FT, SC-5	test_mini	100	880	2440	4545	0.0	1.49	0.00
9B FT, SC-5	OOD-1	40	747	1819	3115	0.0	1.20	0.00
9B FT, SC-5 + retry	test_mini	100	880	2440	4545	0.0	1.53	0.00
9B FT, SC-5 + retry	novel_topo	564	893	2590	4325	0.0	1.45	0.00
9B FT, SC-5 + retry	OOD-1	40	747	2176	4544	0.0	1.20	0.85
9B FT + syn	test	1000	874	479	940	0.0	0.28	0.00
9B FT + syn	test_anon	1000	865	479	940	0.0	0.27	0.00
9B FT + syn	novel_topo	564	893	518	865	0.0	0.29	0.00
9B FT + syn	OOD-1	40	747	395	725	0.0	0.59	0.00
9B FT + syn	OOD-2	21	670	230	349	0.0	0.51	0.00
9B FT + syn + retry	test	1000	874	479	940	0.0	0.28	0.00
9B FT + syn + retry	novel_topo	564	893	518	865	0.0	0.28	0.00
9B FT + syn + retry	OOD-1	40	747	434	1890	0.0	0.59	0.07
9B FT + syn + retry	OOD-2	21	670	333	960	0.0	0.51	0.38

B.15 Determinism and small inconsistencies

Every run used greedy decoding (temperature 0) and seed 643, except self-consistency (temperature 0.7) and thinking mode (temperature 0.6, top-p 0.95, top-k 20, Qwen's recommended settings). Greedy decoding in vLLM is *not* bit-identical across runs when the set of prompts in a batch differs: GPU kernels add numbers in a batch-dependent order, and a tiny difference can flip one token, after which the answer diverges. This explains the few places where two runs that should agree differ by one or two samples: FT-2B + synthetic on OOD-2 is 71.4% in its own run but 66.7% as the first attempt of the retry run; FT-9B is 99.7% on test, while the first attempt inside the FT-9B retry run is 99.6%; FT-2B + retry ends at 93.2% on test, below the 93.3% of the run without retry. These differences are one or two samples and well inside the confidence intervals. The 9B 3-shot unconstrained run (54.6%) vs constrained (54.9%) is a difference of the same size.

C Data details

C.1 The given dataset, per tier

Tier	samples	train	test	nodes min/med/max	edges min/med/max	structures	kind-labelled
L0	1000	800	200	8/10/14	8/11/16	6	18
L1	1000	800	200	6/15/26	5/14/25	21	238
L2	1000	800	200	9/17/26	9/18/28	14	14
L3	1000	800	200	8/12/20	8/13/22	12	12
L4	1000	800	200	15/17/19	16/18/20	4	4

Table 18: Per-tier statistics of the 5,000 samples. "Structures" ignore names and kinds (54 in total); "kind-labelled" structures also distinguish node kinds (286 in total; L1 varies most because its activations and normalisations vary). Every one of the 53 structures in the official test also occurs in the official train split.

C.2 Splits and the novel-topology holdout

Split	samples	per tier (L0/L1/L2/L3/L4)	purpose
train_ft	3068	–	fine-tuning data: official train minus held-out structures minus val
val	160	36 / 36 / 28 / 32 / 28	validation (5% of families; loss curve and sanity check only)
novel_topo	564	112 / 92 / 120 / 120 / 120	official-train samples whose structure was held out of train_ft
test	1000	200 / 200 / 200 / 200 / 200	official test, unchanged
test_anon	1000	200 / 200 / 200 / 200 / 200	official test, every class renamed to `Net`
test_mini	100	20 / 20 / 20 / 20 / 20	stratified 100-sample subset of test (expensive experiments)
test_struct_unseen	200	–	the official-test samples whose structure was held out

Table 19: Splits (seed 643; all ids are stored in `data/processed/splits.json`).

Tier	structure classes in train	held out	samples per held-out class
L0	6	1	112
L1	21	4	4, 12, 36, 40
L2	14	3	64, 84, 56
L3	12	2	72, 64
L4	4	1	228

Table 20: The novel-topology holdout: 11 structure classes chosen with a fixed seed (never the largest class of a tier). All samples with these structures were removed from train_ft.

C.3 Node kinds, token budget, few-shot examples

Most frequent node kinds (node counts over all 5,000 graphs): Conv2d 21808, relu 16924, BatchNorm2d 6664, placeholder 5000, output 5000, Linear 4424, cat 3000, add 3000, sigmoid 2668, GELU 1508, Dropout2d 1324, tanh 1128. The full vocabulary is small: the dataset uses a handful of layer types.

Token audit (Qwen3.5 tokenizer). Training examples (prompt + answer): mean 1333, 99th percentile 2087, maximum 2157 tokens over 3,068 samples; validation maximum 1955. The training `max_length` is 2,240, so no example was truncated. Three-shot prompts are up to 3,107 tokens (mean 2726). The answer ends with the turn-end token `<|im_end|>` (id 248046); the 0.8B, 2B and 9B chat templates are byte-identical, so one set of training files serves all sizes.

Few-shot examples (always the same three, from train_ft, original variant): `L0_0712`, `L2_0560`, `L4_0432` (one L0, one L2, one L4). They are never evaluation samples.

C.4 OOD-1, OOD-2 and the untraceable modules

Table 21: OOD-1 modules (source code in `data/ood/`). The validator reports zero errors on every gold graph.

Module	category	nodes	input shape	code lines
`OOD_bottleneck_1`	bottleneck	14	[1, 16, 16, 16]	18
`OOD_bottleneck_2`	bottleneck	10	[1, 24, 16, 16]	13
`OOD_bottleneck_3`	bottleneck	17	[1, 8, 16, 16]	16
`OOD_bottleneck_4`	bottleneck	12	[1, 32, 16, 16]	15
`OOD_dense_1`	densenet	11	[1, 8, 16, 16]	14
`OOD_dense_2`	densenet	10	[1, 6, 16, 16]	13
`OOD_dense_3`	densenet	11	[1, 4, 16, 16]	15
`OOD_dense_4`	densenet	13	[1, 10, 16, 16]	14
`OOD_inception_1`	inception	18	[1, 16, 16, 16]	19
`OOD_inception_2`	inception	10	[1, 8, 16, 16]	15
`OOD_inception_3`	inception	16	[1, 3, 16, 16]	19

Module	category	nodes	input shape	code lines
OOD_inception_4	inception	12	[1, 24, 16, 16]	17
OOD_mlp_ln_1	mlp_norm	7	[1, 12]	13
OOD_mlp_ln_2	mlp_norm	9	[1, 20]	16
OOD_mlp_ln_3	mlp_norm	9	[1, 8]	14
OOD_mlp_ln_4	mlp_norm	10	[1, 6]	14
OOD_modulelist_1	modulelist	9	[1, 8]	10
OOD_modulelist_2	modulelist	15	[1, 4, 16, 16]	11
OOD_modulelist_3	modulelist	9	[1, 3, 16, 16]	15
OOD_modulelist_4	modulelist	9	[1, 6]	11
OOD_resnet_nested_1	nested_basicblock	19	[1, 3, 16, 16]	27
OOD_resnet_nested_2	nested_basicblock	13	[1, 8, 16, 16]	20
OOD_resnet_nested_3	nested_basicblock	21	[1, 3, 16, 16]	27
OOD_resnet_nested_4	nested_basicblock	18	[1, 16, 16, 16]	24
OOD_se_1	squeeze_excite	12	[1, 16, 16, 16]	13
OOD_se_2	squeeze_excite	11	[1, 8, 16, 16]	13
OOD_se_3	squeeze_excite	13	[1, 12, 16, 16]	14
OOD_se_4	squeeze_excite	17	[1, 24, 16, 16]	15
OOD_two_input_1	two_input	9	[1, 3, 16, 16] + [1, 3, 16, 16]	11
OOD_two_input_2	two_input	11	[1, 3, 16, 16] + [1, 5]	12
OOD_two_input_3	two_input	11	[1, 10] + [1, 6]	13
OOD_two_input_4	two_input	12	[1, 4, 16, 16] + [1, 4, 16, 16]	13
OOD_unet_1	unet	12	[1, 3, 16, 16]	16
OOD_unet_2	unet	12	[1, 1, 16, 16]	15
OOD_unet_3	unet	19	[1, 4, 16, 16]	21
OOD_unet_4	unet	10	[1, 2, 16, 16]	13
OOD_vgg_1	vgg	10	[1, 3, 16, 16]	14
OOD_vgg_2	vgg	12	[1, 1, 16, 16]	15
OOD_vgg_3	vgg	14	[1, 4, 16, 16]	17
OOD_vgg_4	vgg	10	[1, 3, 16, 16]	15

Table 22: OOD-2 modules (source code in `data/ood2/`). The validator reports zero errors on every gold graph.

Module	category	nodes	input shape	code lines
OOD2_attn_1	matmul_attention	13	[1, 8]	14
OOD2_attn_2	matmul_attention	6	[1, 12]	10
OOD2_attn_3	matmul_attention	11	[1, 4, 16, 16]	11
OOD2_deconv_1	transposed_conv	6	[1, 3, 16, 16]	8
OOD2_deconv_2	transposed_conv	9	[1, 4, 16, 16]	11
OOD2_deconv_3	transposed_conv	10	[1, 1, 16, 16]	13
OOD2_fpool_1	functional_pool_reshape	10	[1, 3, 16, 16]	12
OOD2_fpool_2	functional_pool_reshape	7	[1, 4, 16, 16]	11
OOD2_fpool_3	functional_pool_reshape	6	[1, 2, 16, 16]	11
OOD2_mixed_1	mixed_functional	11	[1, 3, 16, 16]	12
OOD2_mixed_2	mixed_functional	8	[1, 12]	9
OOD2_mixed_3	mixed_functional	6	[1, 2, 16, 16]	9
OOD2_norm_1	groupnorm_prelu	6	[1, 8, 16, 16]	9
OOD2_norm_2	groupnorm_prelu	7	[1, 6, 16, 16]	11
OOD2_norm_3	groupnorm_prelu	8	[1, 16]	12
OOD2_scalar_1	scalar_arith	5	[1, 6, 16, 16]	7
OOD2_scalar_2	scalar_arith	8	[1, 5]	9
OOD2_scalar_3	scalar_arith	8	[1, 4, 16, 16]	8
OOD2_stack_1	stack_reduce	7	[1, 4, 16, 16]	10
OOD2_stack_2	stack_reduce	7	[1, 8]	10
OOD2_stack_3	stack_reduce	5	[1, 3, 16, 16]	8

Module	construct that breaks tracing	torch.fx error
UNT_assert_value	an `assert` on a tensor value	TraceError
UNT_data_dependent_if	an `if` on a tensor value	TraceError
UNT_data_dependent_loop_count	a loop count computed from a tensor	TypeError
UNT_early_exit	a `return` inside a data-dependent `if`	TraceError
UNT_int_shape	`int(...)` of a traced shape value	TypeError
UNT_while_loop	a `while` loop on a tensor condition	TraceError

Table 23: The six untraceable modules. There is no reference graph; they are used only to show what the models do (Example 12).

C.5 The synthetic data generator

`scripts/gen_synthetic.py` builds random modules from these building blocks, each with random channel counts (4–32), random kernel sizes and random attribute names:

- **units**: convolution (optionally strided) with optional batch norm and one of several activations (module or functional form); dropout; pooling (max/avg);
- **blocks**: residual (identity or 1×1 projection skip), branch-and-concatenate (2–3 branches), U-Net skip (down-sample, process, up-sample, concatenate), dense block (repeated concatenation), squeeze-and-excitation gating (`mean`, `Linear`, `sigmoid`, broadcasting multiply), `nn.Sequential` stacks, `nn.ModuleList` loops, nested block classes (attribute paths like `stage1.conv`);
- **heads and bodies**: convolutional head (pool, flatten, `Linear`), MLP bodies with LayerNorm, GELU and Dropout, two-input fusion models;
- **noise**: random class names (`Net`, `Encoder`, `Tower`, ...), random forward-argument names (`x`, `inp`, `feats`, ...), dead layers, shuffled `__init__` order.

A module is kept only if torch.fx traces it, the validator reports zero errors on its gold graph, it has at most 40 nodes and fits the token budget, and its structure is not one of the novel_topo or OOD-1 structures (also not an OOD-1 graph's kind-labelled structure). Numbers: 3,200 generated with seed 6430 (264 candidates rejected for a protected structure on the way), 2,090 distinct structures, 2,809 within the token budget and added to training (5,877 training samples in total). The leak audit (`scripts/audit_synthetic.py`) found zero novel_topo or OOD-1 graphs sharing a structure with any trained synthetic module; the closest source-code pair has a token Jaccard similarity of 0.66 (median of the closest pairs 0.53) and is visibly different code. OOD-2 was written after the generator was frozen.

D Prompts and output format

D.1 The system prompt (identical for every model, training and inference)

```
You convert PyTorch nn.Module source code into the forward-pass dataflow graph that
torch.fx.symbolic_trace would produce after dead-code elimination.

Rules:
1. Nodes are listed in execution order: one input node {"op":"placeholder","kind":"placeholder"} per forward()
   input, then one node per executed operation, then exactly one final node
   {"name":"output","op":"output","kind":"output"}. If forward() has a single input, name its placeholder "x"
   regardless of the argument name; with several inputs, use the argument names.
2. self.<attr>(...) on a submodule -> op "call_module", name = attribute path with "." replaced by "_",
   kind = the module class name (e.g. "Conv2d", "BatchNorm2d", "ReLU", "Linear").
3. torch functions and torch.nn.functional calls (torch.relu, torch.cat, torch.flatten, F.relu, ...) and
   Python operators on tensors (a + b -> "add", a * b -> "mul") -> op "call_function",
   kind = the function name ("relu", "cat", "add", ...).
4. Tensor methods (x.view(...), x.flatten(1), x.mean(...)) -> op "call_method", kind = the method name.
5. Node names are globally unique: the first use of a name is plain ("relu"), later ones get suffixes
   "relu_1", "relu_2", ... in execution order (this also applies when a module attribute is called twice).
6. Skip anything that does not contribute to the returned value (dead code). Plain variable aliasing
   (y = x) creates no node.
7. "shape" is the output tensor shape assuming batch size 1. Conv networks receive 16x16 spatial inputs,
   i.e. [1, C_in, 16, 16]; networks whose first layer is Linear receive [1, in_features]. With several
   inputs, each input's shape follows the same rule using the first layer that consumes it.
8. "edges" is a list of [i, j] pairs (indices into "nodes"): the output of node i is a direct input of
   node j, listed in the order node j consumes them. The output node consumes the returned tensor.

Return only the JSON object {"nodes": [...], "edges": [...]} on a single line (compact, no indentation) with no
explanation.
```

D.2 User message, few-shot layout, retry message

User message. The source code inside a Python code block; for the multi-input OOD modules (whose input order rule 7 cannot fix) one extra line gives the input shapes:

````
Source code:
```python
<source code>
```
Input shape: <only for multi-input modules>
````

Few-shot layout. System prompt, then three (user, assistant) pairs with the three fixed training examples and their exact CG-JSON answers, then the real user message. **Retry message** (sent as a new user turn after the model's previous answer; at most 12 error messages, at most 2 rounds):

```
Your graph has these problems:
- <validator message 1>
- <validator message 2>
Return the corrected full JSON graph.
```

Chat rendering. Messages are rendered with each model's own chat template. For Qwen3.5 with thinking off, the template ends the prompt with `<|im_start|>assistant`, a newline and an empty `<think></think>` block, so the model starts directly with the JSON. With thinking on, the model writes its reasoning first; an answer counts only if the reasoning was closed with `</think>`. The training answers are the compact one-line CG-JSON followed by exactly one `<|im_end|>` token (checked on the trainer's own batch).

D.3 The JSON schema used for constrained decoding

```
{
  "type": "object",
  "properties": {
    "nodes": {"type": "array", "minItems": 2, "items": {
      "type": "object",
      "properties": {
        "name": {"type": "string", "pattern": "^[A-Za-z_][A-Za-z0-9_]*$"},
        "op": {"type": "string", "enum": ["placeholder","call_module","call_function","call_method","get_attr","
            output"]},
        "kind": {"type": "string"},
        "shape": {"type": "array", "items": {"type": "integer", "minimum": 1}}
      },
      "required": ["name","op","kind","shape"], "additionalProperties": false}},
    "edges": {"type": "array", "items": {"type": "array", "items": {"type": "integer", "minimum": 0}, "minItems
        ": 2, "maxItems": 2}}
  },
  "required": ["nodes","edges"], "additionalProperties": false
}
```

Parser (`tgraph/parse.py`), applied to every raw answer: remove a `<think>` block (for thinking runs, no `</think>` means no answer), strip code fences, take the first balanced `{...}`, parse it, and if that fails repair it (close brackets of a truncated answer) with `json_repair`; numbers written as strings (`"16"`) are converted; then the schema is checked. Constrained runs always parse (the grammar guarantees it), so their 100% parse rate is by construction and not a result.

E Validator and metric reference

E.1 Every validator check

Table 24: All checks in `tgraph/validate.py`. They use only the predicted graph and the source code (read with Python's `ast`, never executed), never the reference. A level passes when none of its errors occur; *fully valid* = V0–V4 all pass. If the code analysis itself fails, V3 is skipped with a warning.

Level	code	what it means / message
V0	PARSE	no JSON object could be extracted
V0	SCHEMA	JSON does not match the schema (e.g. a node without `shape`)
V1	DUP_NAME	node … duplicates the name of node …
V1	BAD_EDGE	edge [a, b] points outside the n nodes

Level	code	what it means / message
V1	SELF_LOOP	edge [a, a] is a self-loop
V1	DUP_EDGE	edge [a, b] is listed twice
V1	NO_INPUT	graph has no placeholder (input) node
V1	OUTPUT_COUNT	graph needs exactly one output node as the last node
V1	INPUT_HAS_PRED	placeholder has incoming edges
V1	OUTPUT_DEGREE	output node needs $\geq$1 input and no outgoing edges
V1	CYCLE	graph has a cycle through nodes ...
V1	ORDER	edge goes backwards: nodes must be listed in execution order
V1	ORPHAN	node is not reachable from any input
V1	DEAD_END	node does not lead to the output (dead end or dead code)
V1	OP_KIND	op and kind disagree (e.g. op `placeholder` with kind `input`)
V2	ARITY	wrong number of inputs: binary functions such as `add`/`mul` need 2 tensors (one input is only a warning if the code contains a tensor–scalar operation); joining functions such as `cat` need $\geq$2; single-input functions and layers exactly 1; the output node exactly 1 (unless `forward` returns a tuple)
V3	UNDECLARED_ MODULE	a `call_module` node that is not a layer declared in `__init__` (hallucinated)
V3	KIND_MISMATCH	node has kind X but `self.<attr>` is a Y
V3	MISSING_MODULE	a layer is called k times in `forward` but appears fewer times in the graph
V3	DEAD_CODE_ INCLUDED	a layer whose result never reaches the `return` appears in the graph
V4	SHAPE	input shape differs from the convention; conv/linear output channels differ from `out_channels`/`out_features`; shape-preserving ops change the shape; reshapes change the element count; `add`/`mul` inputs not broadcast-compatible or output not their broadcast; `cat` output not the concatenation

E.2 Metric definitions

Table 25: Per-sample metrics (`tgraph/metrics.py`); a parse failure scores 0 on everything. Before comparing, a single input node is renamed to `x` in both graphs (the dataset convention), so the forward argument's name does not matter.

Metric	definition
node F1	nodes matched by exact name; a matched node is correct if its op and kind also agree; F1 of precision and recall
node F1 (aligned)	same, after also pairing unmatched nodes of the same kind by the Hungarian algorithm (cost = difference in graph depth)
edge F1	edges as (source name, target name) pairs; F1 against the reference edge set
edge F1 (aligned)	edges mapped through the aligned node matching
exact match	same multiset of (name, op, kind) and the same named edge set; shapes are ignored
isomorphic	same graph up to renaming, with node kinds as labels (networkx VF2)
WL-equal	equal Weisfeiler–Lehman hashes (3 iterations, kinds as labels): a fast approximate isomorphism test
shape accuracy	share of name-matched nodes whose shape equals the reference shape
AED	aligned edit distance: unmatched nodes on both sides + matched nodes with a different kind + edges in exactly one of the two graphs (after mapping) + predicted edges touching unmatched nodes
graph similarity	$1 - \min(1, \text{AED}/(\lvert V_p\rvert + \lvert V_r\rvert + \lvert E_p\rvert + \lvert E_r\rvert))$
PPS	property preservation score: share of the 14 properties below whose value equals the reference's
reach F1	F1 of the transitive reachability pairs ($u \rightsquigarrow v$) under the aligned matching: rewards connectivity even when one edge is misplaced
kind / typed-edge multiset F1	F1 of the multisets of node kinds and of (kind, kind) edge pairs: name-free
fully valid	passes validator levels V0–V4 (no reference needed)

Property	definition
n_nodes, n_edges	number of nodes / edges
n_sources, n_sinks	nodes without inputs / outputs
n_branch, n_merge	nodes with out-degree >1 / in-degree >1
depth	longest path from an input (in levels)
width	largest topological generation
n_skip_edges	edges that jump over at least one level (residual and dense connections)
n_paths_in_out	number of input-to-output paths (capped at 10,000)
n_cycles	number of directed cycles (0 for a valid graph; capped at 100)
hierarchy_groups	name prefixes shared by at least two layer nodes (`fc1_0`, `fc1_1`, ...; or dotted scopes)
is_dag, weakly_connected	acyclic / connected ignoring direction

Table 26: The 14 topological properties (`tgraph/properties.py`).

Label	rule
parse_fail	no graph could be extracted
missing_skip_edge	a reference edge that jumps >1 level is absent
missing_edge	a reference edge between adjacent levels is absent
extra_edge	a predicted edge that is not in the reference (or touches an unmatched node)
missing_node	a reference node has no counterpart
hallucinated_node	a predicted node has no counterpart
dead_code_included	the validator found a dead layer in the prediction
wrong_kind	a matched node has the wrong kind (e.g. `ReLU` vs `relu`)
naming_only	the graph is isomorphic to the reference but some names differ
shape_only	exact topology, but some shapes are wrong
order_error	edges go backwards in the node list
cycle	the graph has a cycle

Table 27: Failure labels (`tgraph/failures.py`); a sample can get several.

F Training and inference settings

Setting	value
method	supervised fine-tuning with LoRA (bf16, not quantised), TRL `SFTTrainer`; loss only on the answer tokens
LoRA	rank 32, alpha 64, dropout 0.05
LoRA targets	`q_proj k_proj v_proj o_proj gate_proj up_proj down_proj` (attention and MLP) and `in_proj_qkv in_proj_z out_proj` (Gated DeltaNet linear-attention layers); not `in_proj_a/b` (one value per head), not the vision tower, not the multi-token-prediction head
trainable parameters	2B: 31.3 M; 9B: 80.2 M (checked: 0 in the vision tower)
schedule	2 epochs, cosine learning-rate decay, 3% warm-up; learning rate $2 \cdot 10^{-4}$ (2B) and 10^{-4} (9B)
batch	effective 16: 2B 8 per step × 2 accumulation; 9B 2 × 8
sequence	`max_length` 2,240 tokens, no packing, no truncation; gradient checkpointing (non-reentrant)
checkpoints	evaluation and save every 50 steps, last 2 kept; seed 643
data	train_ft (3,068) or train_ft + 2,809 synthetic (5,877); validation 160
merge	`scripts/merge_lora.py` adds $\frac{\alpha}{r}BA$ to each adapted weight directly in the safetensors files, so the merged folder has the base model's exact layout (vLLM loads it like the base model). Checked on 0.8B with random adapters: merged and adapter logits agree within 0.1 (vs an adapter effect of 11.7)
sanity check	after training, 20 validation samples are decoded greedily: 20/20 exact for all four adapters

Table 28: Fine-tuning settings (`scripts/train_sft.py`, `configs/train_*.yaml`).

Adapter	steps	time	s/step	first loss	last loss	val loss
9B + synthetic	736	3.00 h	14.7 s	0.050	8.0e-06	8.3e-07
9B	384	1.48 h	13.9 s	0.027	2.4e-06	1.4e-06
2B + synthetic	736	1.37 h	6.7 s	0.106	7.9e-05	2.8e-06
2B	384	0.50 h	4.7 s	0.058	2.8e-05	2.7e-05

Table 29: Training runs (one NVIDIA L40S each), from `runs/train_*/log_history.csv`. Losses near zero are expected: the answer is fully determined by the code.

Setting	value
engine	vLLM 0.26.0, bf16 weights (27B: the official FP8 checkpoint), one GPU
context	`max_model_len` 8,192 (thinking: 12,288 or 34,816)
answer budget	3,072 tokens (thinking: 8,192 or 30,000)
decoding	greedy, seed 643; self-consistency T = 0.7; thinking T = 0.6, top-p 0.95, top-k 20
constrained decoding	vLLM structured outputs with the JSON schema above; retry rounds are always constrained; for think-then-constrain the grammar applies only after `</think>` (`reasoning_parser=qwen3`)
memory	`gpu_memory_utilization` 0.88 (27B: 0.92 and at most 64 parallel sequences, because Qwen3.5's recurrent-state cache left room for only 151 sequences next to the 27B weights)
other	prefix caching off (vLLM marks it experimental for Qwen3.5's linear-attention layers); multimodal inputs disabled; vLLM's PyTorch sampler (the default flashinfer sampler needs a CUDA compiler the cluster lacks)
batching	all prompts of an evaluation set in one call; retry rounds re-submit only the invalid answers
HF backend	used by the app's CPU mode: transformers `generate`, greedy, no constrained decoding

Table 30: Inference settings (`scripts/generate.py`).

G Software, hardware and compute

Package	tg-train	tg-infer
Python	3.11	3.11
PyTorch	2.11.0+cu128	2.11.0+cu129
transformers	5.19.0	5.20.2
TRL / PEFT / accelerate	1.16.0 / 0.21.2 / 1.15.0	– / 0.21.2 / 1.15.0
flash-linear-attention	0.5.2	–
vLLM	–	0.26.0+cu129
xgrammar / flashinfer	–	0.2.7 / 0.6.14
torchcodec	–	0.15.0+cu129 (+ conda ffmpeg <8)
networkx / scipy / numpy	3.6.1 / 1.17.1 / 2.4.6	3.6.1 / 1.17.1 / 2.3.5
gradio / matplotlib / graphviz	6.30.0 / 3.11.2 / 0.21	6.30.0 / 3.11.2 / 0.21
json_repair / jsonschema	0.64.2 / 4.26.0	0.64.2 / 4.26.0

Table 31: The two conda environments (exact pins: `requirements-*.lock.txt`). Two are needed because vLLM pins its own PyTorch build.

Hardware and cluster. IIT Bombay's Prajna cluster: NVIDIA L40S GPUs (48 GB, 44.4 GiB usable), driver 570.86.15 (CUDA $\leq$ 12.8), Lustre file system, no internet on compute nodes (every job runs with the Hugging Face offline flags). SLURM rules that shaped the work: `-qos` must equal the partition; the `l40` partition allows 4 running and 5 submitted jobs per user (two inference jobs were merged for that reason); only the `interactive` partition accepts `srun` (used for CPU work and the live demo; node `cn22-a40` was excluded after "Job credential expired" errors); compute nodes cannot be reached by `ssh` (the app is reached by port forwarding through the login node).

Isolation. The same account runs another research project whose evaluation depends on bit-exact caches. All of this project's files live under `~/IE643_Project`: `tgenv.sh` redirects the conda package cache, `PIP_CACHE_DIR`, `HF_HOME`, `TRITON_CACHE_DIR`, `TMPDIR`, `XDG_CACHE_HOME`,

`VLLM_CACHE_ROOT`, `FLASHINFER_WORKSPACE_BASE`, `TORCHINDUCTOR_CACHE_DIR` and `TORCH_HOME` there. The base Qwen3.5-9B weights are shared read-only through a symbolic link; all other models were downloaded into `models/`.

Table 32: Every SLURM job of the project (`sacct`). Total GPU time 20.6 GPU-hours (all on single L40S GPUs; this includes failed, cancelled and demo jobs), CPU-only jobs 1.3 h on 16 cores. Cancelled jobs are the restart after the prompt fix (Section 4), a ceiling attempt that needed a memory setting, and test runs. `tg-app` is the live-demo test (ended by its 1-hour limit).

Job	name	partition	resources	state	elapsed	start	node
363532	tg-cpusmoke	interactive	CPU	FAILED	00:00:00	10-24 18:21	cn22-a40
363534	tg-cpusmoke	interactive	CPU	FAILED	00:00:00	10-24 18:22	cn22-a40
363535	tg-cpusmoke	interactive	CPU	FAILED	00:05:17	10-24 18:23	cn11-dgx
363542	tg-cpusmoke	interactive	CPU	FAILED	00:00:35	10-24 18:31	cn11-dgx
363554	tg-cpusmoke	interactive	CPU	FAILED	00:01:55	10-24 18:57	cn11-dgx
363569	tg-cpusmoke	interactive	CPU	CANCELLED	00:22:26	10-24 19:15	cn11-dgx
363587	tg-smoke	l40	1 GPU	FAILED	00:10:04	10-24 19:56	cn44-l40
363597	tg-cpusmoke	interactive	CPU	CANCELLED	00:28:05	10-24 19:41	cn11-dgx
363637	tg-smoke	l40	1 GPU	COMPLETED	00:20:25	10-24 20:08	cn44-l40
363789	tg-base-p0	l40	1 GPU	CANCELLED	00:18:23	10-24 23:21	cn43-l40
363790	tg-base-p1	l40	1 GPU	CANCELLED	00:18:23	10-24 23:21	cn41-l40
363791	tg-train-2b	l40	1 GPU	CANCELLED	00:18:23	10-24 23:21	cn41-l40
363792	tg-train-9b	l40	1 GPU	COMPLETED	01:43:07	10-24 23:49	cn41-l40
363815	tg-base-p0	l40	1 GPU	COMPLETED	01:50:27	10-24 23:49	cn41-l40
363816	tg-base-p1	l40	1 GPU	COMPLETED	00:51:26	10-24 23:49	cn44-l40
363817	tg-train-2b	l40	1 GPU	COMPLETED	00:38:46	10-24 23:49	cn43-l40
363818	tg-ftinfer	l40	1 GPU	COMPLETED	00:58:44	10-25 01:32	cn41-l40
363875	tg-ftinfer-2b	l40	1 GPU	COMPLETED	00:12:59	10-25 00:28	cn42-l40
363885	tg-think32k	l40	1 GPU	COMPLETED	01:08:46	10-25 00:41	cn42-l40
363911	tg-sc5	l40	1 GPU	COMPLETED	00:07:58	10-25 02:03	cn41-l40
363912	tg-thinkc	l40	1 GPU	COMPLETED	01:36:24	10-25 02:03	cn42-l40
363915	tg-aug9b	l40	1 GPU	COMPLETED	03:35:34	10-25 02:13	cn41-l40
363916	tg-aug2b	l40	1 GPU	COMPLETED	01:41:29	10-25 02:13	cn42-l40
363923	tg-combo	l40	1 GPU	COMPLETED	00:28:05	10-25 02:32	cn41-l40
363925	tg-ceiling	l40	1 GPU	CANCELLED	00:07:50	10-25 03:00	cn41-l40
363928	tg-ceiling	l40	1 GPU	COMPLETED	01:02:38	10-25 03:08	cn41-l40
363936	tg-ceilretry	l40	1 GPU	COMPLETED	00:37:04	10-25 03:40	cn41-l40
363950	tg-ood2	l40	1 GPU	COMPLETED	00:55:33	10-25 04:00	cn42-l40
363954	tg-ceilfsretry	l40	1 GPU	COMPLETED	00:15:23	10-25 04:11	cn41-l40
363956	tg-aug9b-extra	l40	1 GPU	COMPLETED	00:20:42	10-25 05:49	cn44-l40
363997	tg-cpueval	interactive	CPU	COMPLETED	00:04:56	10-25 06:10	cn11-dgx
364212	tg-app	interactive	1 GPU	TIMEOUT	01:00:02	10-25 13:58	cn41-l40
364274	tg-cpucheck	interactive	CPU	CANCELLED	00:02:09	10-25 15:23	cn11-dgx
364277	tg-cpucheck	interactive	CPU	COMPLETED	00:10:18	10-25 15:25	cn11-dgx

H Decision log

Table 33: Every deviation from (or choice within) the original build spec, condensed from `DECISIONS.md`.

#	decision	what	why
1	Environments and caches	two new conda envs and every cache inside the project folder; the 9B weights reused read-only	do not disturb the other project on the same account
2	vLLM 0.26.0 + CUDA 12.9 build	newest vLLM with a PyTorch build that runs on the CUDA-12.8 driver	vLLM $\geq$0.27 needs PyTorch 2.13 (no CUDA-12.8 build)
3	training env: PyTorch 2.11 cu128 + fla 0.5.2	combination already known to run Qwen3.5 on this cluster; `causal-conv1d` not installed (needs a CUDA compiler)	lowest risk
4	models as plain folders	downloaded with curl into `models/<name>`, loaded by path	compute nodes are offline
5	54 structures, not $\sim$68	the spec double-counted structures shared between tiers	measured with networkx
6	novel-topology holdout	1/4/3/2/1 classes per tier held out (seed 643); 564 samples	test generalisation to unseen topologies
7	`max_length` 2,240	the longest training example is 2,157 tokens	no truncation
8	canonical prompt	ends with an empty think block (template behaviour with thinking off)	byte-identical train/inference prompts

#	decision	what	why
9	oracle sandbox	one long-lived spawned worker with a 20 s per-call limit	importing torch takes >20 s on Lustre; `fork` deadlocks
10	OOD-1 set	40 modules in 10 categories; `se_3` got a conv stem	rule 7 was ambiguous without it
11	validator: scalar arithmetic	`add`/`mul` with one tensor input is an error unless the code has a tensor–scalar operation (then a warning)	zero false alarms on 5,000 gold graphs
11b	prefix caching off	vLLM marks it experimental for Qwen3.5's linear-attention layers	results must not depend on it
12	failure label `missing_edge`	added next to `missing_skip_edge`	a dropped chain edge had no label (found in the dry run)
13	prompt identity across envs	checked 200/200 identical prompts between the two transformers versions	training and inference use different envs
14	prompt v2	rule 1 states the input node's kind `placeholder`; compact one-line JSON; 3,072-token budget; everything re-run	957/1,000 zero-shot answers failed only on that kind; 12% ran out of tokens
15	thinking mode	an answer needs `</think>`; added a 30k-token run	97% unfinished at 8k; the parser had taken fragments from unfinished reasoning
16	oracle warm-up	the worker traces a tiny model before reporting ready; untraceable set skipped in bulk evaluation	first-use imports counted against the 20 s limit
17	night plan	think-then-constrain, self-consistency, the synthetic-data arm	the remaining weakness was generalisation
18	ceiling = Qwen3.5-27B-FP8	official FP8 checkpoint (29 GB) fits one L40S; bf16 (55.6 GB) does not	same family as the main model
19	OOD-2	21 modules written after the generator was frozen, with operators absent from dataset and generator; 3 tuple-output modules dropped	an honest generalisation test for the synthetic-data models

I Problems encountered and how they were fixed

Table 34: Problems met during the project, in order (details in `PROJECT_LOG.md`).

Problem	cause	fix
two conda builds at once	both failed (`InvalidArchiveError`): they share one package cache	rebuilt one after the other
`import vllm` (1)	torchcodec from PyPI is built for CUDA 13	torchcodec 0.15.0+cu129
`import vllm` (2)	torchcodec needs FFmpeg shared libraries	conda-forge `ffmpeg<8`
`import vllm` (3)	FFmpeg's ICU needs a newer C++ runtime than the system's	`LD_PRELOAD` of the env's `libstdc++`
vLLM engine start	flashinfer's sampler compiles CUDA code at run time; no compiler on the cluster	`VLLM_USE_FLASHINFER_SAMPLER=0`
cache leak	vLLM and flashinfer wrote into `~/.cache`	moved into the project; extra cache variables in `tgenv.sh`
Triton on CPU	transformers picks the fla GPU kernels even without a GPU	`cpu_disable_fla()` on CPU-only runs
CPU training	one step of 870 tokens did not finish in 25 min	all training on GPUs
TRL/transformers API changes	`warmup_ratio` and `logging_dir` removed; duplicate end token	`warmup_steps=0.03`, env variable for logs, `eos_token=<|im_end|>`
node cn22-a40	"Job credential expired" on every launch	excluded
oracle subprocess	a fresh process per call exceeded 20 s; `fork` deadlocked	one persistent spawned worker, with warm-up
dry run	a dropped ordinary edge got no failure label	added `missing_edge`
prompt v1	0% zero-shot validity (input-node kind; pretty-printed JSON ran out of tokens)	prompt v2 with the user's OK; all jobs restarted
job limits	`l40` allows only 5 submitted jobs per user	two inference jobs merged, with a dependency on both trainings
thinking answers	8k tokens too few; fragments parsed from unfinished reasoning	`</think>` required, `scripts/reparse.py`, 30k-token run
27B ceiling	default 256 parallel sequences exceeded the recurrent-state cache	`max_num_seqs` 64, memory 0.92
OOD `se_3`	the input-shape rule was ambiguous	added a conv stem
tuple outputs	`chunk`/`split` results have no tensor shape in the schema	3 OOD-2 modules dropped
untraceable metrics	no reference, so no metrics	skipped in bulk evaluation; shown qualitatively
app first click	about 90 s (oracle worker start + grammar compilation)	`-preload` sends one warm-up request

J Timeline

When (IST)	what
24 Oct, afternoon	plan read; environments built; models downloaded; core package, OOD-1 and tests written
24 Oct, evening	all CPU tests green (oracle 5,000/5,000, validator silent on gold); vLLM import repairs; CPU smoke
24 Oct, 20:05–22:30	GPU smoke (two attempts): vLLM, training speed (9B 12.1 s/step), merge check, evaluation dry run
24 Oct, 23:21	full runs submitted on the user's "submit" (baselines, 2B and 9B training)
24 Oct, 23:40	prompt fix (v2) after the first zero-shot results; all affected jobs restarted
25 Oct, 00:45	first results: 9B 3-shot 54.9%, FT-2B 93.3% (0% on the held-out L4 topology), thinking needs more tokens
25 Oct, 02:00–03:30	FT-9B 99.7% on test but 47.5% on OOD-1; night plan: synthetic data, 27B ceiling, retry, self-consistency, think-then-constrain
25 Oct, 03:50	OOD-2 written after the generator was frozen
25 Oct, 06:07	last GPU job finished
25 Oct, 06:40	final evaluation on CPU: metrics, GED check, tables, figures, galleries, app cache
25 Oct, daytime	explainer, live GPU demo test, README, model card, CPU-mode and adapter checks, submission zip

Table 35: Project timeline.

K Repository inventory and tests

Table 36: Every code and configuration file (descriptions are the files' own first comment lines).

File	purpose
`app/app.py`	TopoGraph Gradio app (build spec ğ14).
`app/core.py`	Core handler for the TopoGraph app (UI-free, so it can be tested directly).
`scripts/aggregate.py`	Aggregate per-sample metrics into tables (build spec ğ13). Bootstrap 95% CIs (1,000 resamples).
`scripts/audit_synthetic.py`	Leakage audit: trained synthetic modules vs novel_topo/OOD (structure overlap, closest source-token Jaccard).
`scripts/build_app_cache.py`	Store predictions for the app's examples in app/precomputed.jsonl so "Precomputed" mode works from a fresh clone (results/raw is not shipped). python scripts/build_app_cache.py
`scripts/build_ood.py`	Label the hand-written OOD modules with the oracle (build spec ğ7.2) and write data/processed/{ood,untraceable}.jsonl.
`scripts/build_sft_aug.py`	sft_train_aug.jsonl = sft_train.jsonl (official train_ft) + rendered synthetic modules (same prompt/serialisation).
`scripts/failure_analysis.py`	Failure taxonomy summary (build spec ğ9.3): counts per config x tier and 2 example ids per label.
`scripts/gallery.py`	Qualitative gallery (build spec ğ10): per tier best/median/worst on test, OOD samples, untraceable examples.
`scripts/ged_subset.py`	True GED vs AED on a stratified 100-sample subset of test per config (build spec ğ9.2, figure F8).
`scripts/gen_synthetic.py`	Synthetic training modules with new topologies, labelled by the torch.fx oracle (night experiment "aug").
`scripts/generate.py`	Inference (build spec ğ12): vLLM (GPU, default) or HF transformers backend, optional constrained decoding, verify-and-retry, self-consistency, thinking mode. Resumable: finished (config, eval set) files are skipped.
`scripts/merge_lora.py`	Merge a LoRA adapter into the base weights at the safetensors level (build spec ğ11).
`scripts/plots.py`	Figures F1-F10 (build spec ğ13) from results/tables + per-sample metrics. PNG + SVG.
`scripts/prepare_data.py`	Data preparation (build spec ğ7): records, structure ids, novel-topology holdout, val split, eval sets, SFT files, token audit, dataset stats. Deterministic with seed 643.
`scripts/reparse.py`	Re-parse and re-validate stored raw outputs with the current parser (no generation).
`scripts/run_eval.py`	Per-sample validity + metrics for prediction files (build spec ğ13). CPU only, multiprocessing.
`scripts/train_sft.py`	LoRA SFT of Qwen3.5 on CG-JSON completions (build spec ğ11). Env: tg-train.
`tgraph/__init__.py`	TopoGraph: LLMs generating torch.fx forward-pass dataflow graphs from nn.Module source.
`tgraph/align.py`	Node alignment between predicted and reference canonical graphs: exact name match, then Hungarian on the rest.
`tgraph/codeinfo.py`	AST analysis of nn.Module source (no execution).

File	purpose
tgraph/compat.py	Runtime switches that must run before transformers loads Qwen3.5's modeling code.
tgraph/failures.py	Rule-based failure taxonomy for one prediction (a sample can get several labels).
tgraph/io.py	JSONL helpers and dataset loading.
tgraph/metrics.py	Per-sample metrics between a predicted and a reference graph (both canonical format).
tgraph/oracle.py	Reference-graph oracle: torch.fx symbolic trace + dead-code elimination + shape propagation.
tgraph/parse.py	Robust extraction of CG-JSON from raw LLM text.
tgraph/prompts.py	System prompt, user template, chat rendering and few-shot exemplars.
tgraph/properties.py	Topological properties of a canonical graph (used for the Property Preservation Score).
tgraph/schema.py	Compact graph JSON (CG-JSON, what the LLM emits) <-> canonical dataset format, plus the JSON schema.
tgraph/validate.py	Graph validity and topological-consistency checker (never uses the reference graph).
tgraph/visualize.py	Graph rendering: graphviz (preferred) with a matplotlib fallback when the 'dot' binary is missing.
jobs/app_gpu.sh	Live GPU demo: the app with vLLM on one GPU, reachable from the login node at http://<node>:7860.
jobs/common.sh	Sourced by every job script: env, offline flags, GPU banner. Usage: source jobs/common.sh train|infer
jobs/cpu_eval.sh	Metrics for every prediction file, then tables, figures, failure analysis, galleries, app cache. CPU only.
jobs/cpu_smoke.sh	CPU smoke (spec ğ11): 0.8B, 16 rows, 3 steps -> merge -> HF generate -> parse -> validate -> metrics -> render.
jobs/gpu_smoke.sh	GPU smoke (spec ğ3.5): CUDA checks, vLLM + HF 2-sample generation with 0.8B, 20 LoRA steps of 9B for timing.
jobs/infer.sh	Generic inference job: CONFIGS="cfg1 cfg2 ..." [UNTRACEABLE="cfgA cfgB"]. Idempotent and fail-soft.
jobs/infer_baselines.sh	P0: E1 E2 E7 E3, then P1: E9 (retry) E16 (thinking) E17 (Coder-7B). Idempotent: finished (config, set) files are
jobs/infer_ceiling.sh	P1: E17 (Coder-7B few-shot) and, if $MODELS/CEILING exists, E10/E11.
jobs/infer_finetuned.sh	P0: E4 E5 E6, P1: E8 (retry), plus untraceable predictions for the gallery/app. Run after train.sh.
jobs/train.sh	2B then 9B LoRA SFT (resumable), then merge both. WHICH=2b|9b|both (default both).
jobs/train_one.sh	One LoRA training + merge + inference: CFG=configs/train_X.yaml BASE=<model dir name> TAG=<adapter tag> INFER="<configs>"
scripts/package.sh	Submission zip: code, configs, splits, OOD sets, synthetic data, results, explainer, LoRA adapters.
scripts/run_app.sh	Launch the TopoGraph app. scripts/run_app.sh –mode gpu|cpu|precomputed [–port 7860]
configs/experiments.yaml	every inference configuration
configs/graph_schema.json	JSON schema of CG-JSON (constrained decoding)
configs/train_2b.yaml	LoRA training configuration
configs/train_2b_aug.yaml	LoRA training configuration
configs/train_9b.yaml	LoRA training configuration
configs/train_9b_aug.yaml	LoRA training configuration
configs/train_smoke.yaml	tiny training run for the smoke tests

Test file	what it checks
test_oracle.py	the oracle reproduces all 5,000 dataset graphs exactly; nested scopes; a data-dependent `if` raises; the sandbox reports errors; two-input models keep argument names
test_schema.py	CG-JSON round trip on all 5,000 graphs; the parser handles fences, think blocks, truncation and prose
test_validate.py	zero errors on every gold graph; each corruption is caught (deleted skip edge, back edge/cycle, undeclared layer, inserted dead code, changed conv shape, dropped output, missing layer and parse failure)
test_metrics.py	identity scores perfectly; corruptions lower the scores; renaming and permutation keep isomorphism; parse failure scores 0; AED is an upper bound of the true GED
test_generate_logic.py	with a fake model: few-shot prompts contain 3 examples, the retry turn contains the validator messages, a fixed graph is kept, self-consistency samples 5
test_visualize.py	both drawing back ends; diff and neuron-expansion drawings
test_app_smoke.py	the app handler: reference-only drawing, friendly errors for bad input, the untraceable message, every stored example renders

Table 37: The test suite (`pytest -q`: 28 tests, all passing at the final commit).

L The demo app

What it shows. Paste or upload an `nn.Module` (or pick one of the stored examples), choose a model, and press *Generate*. Tabs: the predicted graph, the torch.fx reference (if the code is traceable), the colour-coded diff, the metrics and the 14 topological properties side by side, the validity report (pass/fail per level, every message, retry rounds used), and both graphs as JSON (downloadable). Options: constrained decoding, verify-and-retry, drawing small `Linear` layers as neurons, an input-shape override and the class name to use. A *Batch* tab scores a JSONL file of modules and returns a CSV of metrics. The pasted code is executed (in the sandboxed oracle worker, 20 s limit) to build the reference, so the app is a local tool, not a public service.

Modes (`scripts/run_app.sh -mode ...`):

- `precomputed`: no model; the live torch.fx reference plus stored predictions, looked up by a hash of the code: six configurations for every dataset and OOD sample when `results/raw` is present, otherwise the 45 predictions shipped in `app/precomputed.jsonl` (the dropdown examples). Runs on any laptop.
- `cpu`: adds live generation with the Hugging Face backend (no constrained decoding). Measured on 16 server cores with FT-2B + synthetic: 4.4 min for the first graph (including the model load), 2.5 min per graph afterwards, because Qwen3.5's linear-attention layers fall back to plain PyTorch without a GPU. Too slow for a live demo.
- `gpu`: adds live generation with vLLM. The default model is FT-9B + synthetic; FT-9B, FT-2B (+ synthetic), a base 2B with the adapter, and the base 9B with 3 examples are also offered. Each loaded model takes 45% of the GPU, so two fit on one L40S. With `-preload`, the default model is loaded and one warm-up request is sent before the page opens (about 5 minutes); afterwards a graph takes about 9 s.

Running the live demo on Prajna (from a terminal on the login node, ideally inside `tmux`):

```
cd ~/IE643_Project/topograph
srun -p interactive --qos=interactive --gres=gpu:1 -c 8 --mem=64G -t 2:00:00 \
    --exclude=cn22-a40 bash jobs/app_gpu.sh
# wait for 'TopoGraph app ready'; the log prints the node name, then forward <node>:7860:
# VS Code: Ports -> Forward a Port -> <node>:7860
# laptop: ssh -L 7860:<node>:7860 23b1835@login1.prajna.iitb.ac.in -> http://localhost:7860
```

To book a GPU ahead of time instead: `sbatch -p l40 -qos=l40 -c 8 -mem=64G -t 6:00:00 jobs/app_gpu.sh`. **Outside the cluster** (untested): FT-2B fits a free Colab T4; FT-9B needs about 19 GB of GPU memory in bf16 (a Colab L4 is tight, an A100 comfortable). Ship the LoRA adapter (139 MB for 2B, 326 MB for 9B) and either merge it with `scripts/merge_lora.py` or load base model + adapter with PEFT; this was checked to give the identical graph.